F300 Document
Capstone Design
Project Design
Driving SME Growth with AI Based Seller Support Solutions for Selly

GROUP MEMBER:

No.	Student Name	Student ID	Role
1.	Joseph Tedja Nugraha Wibawa	001202200121	Auto Text RAG
2.	Jaya Iskandar MF	001202200042	Personalized Marketing Campaign Generator

Advisor : Dr. Adhi Setyo Santoso, ST., MBA.

Submitted for
Capstone Design Project
to Faculty of Computer Science
President University
January, 2024

TABLE OF CONTENTS

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

Name : 1. Joseph Tedja Nugraha Wibawa – 001202200121

2. Jaya Iskandar MF – 001202200042

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled "**Driving SME Growth with AI Based Seller Support Solutions for Selly**" is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

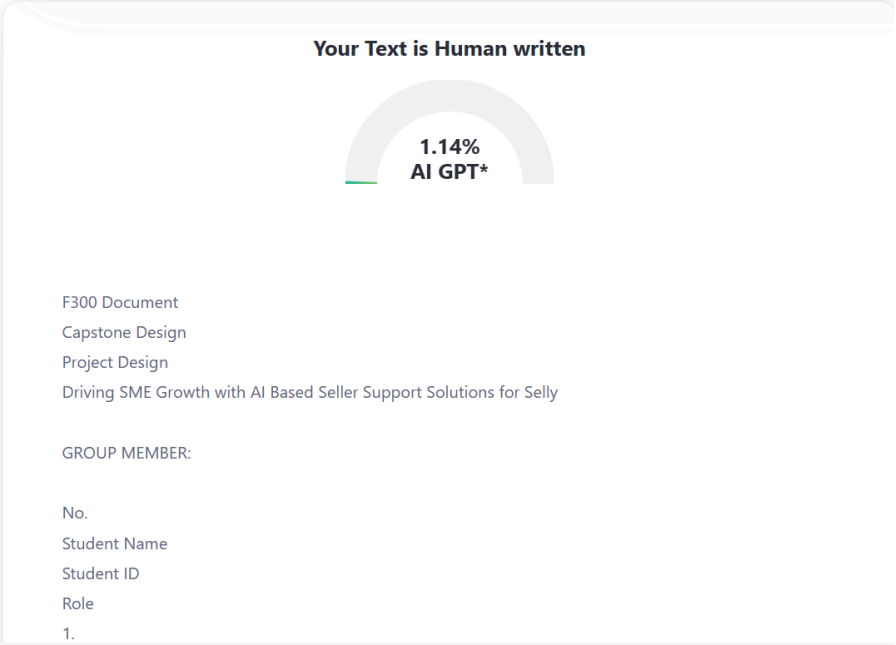
Cikarang, 22 January 2025

Signer 1	Signer 2
	
Joseph Tedja Nugraha Wibawa – 001202200121	Jaya Iskandar MF– 001202200042

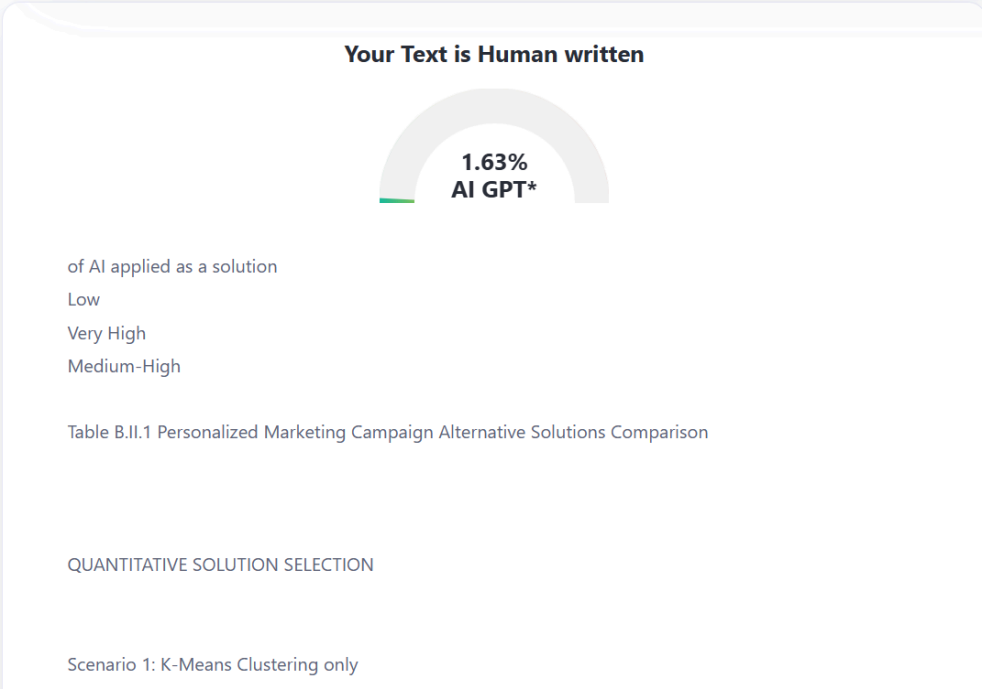
SCREENSHOT OF ZEROGPT

The entire document is 41.168 characters in total, divided by the maximum allowed by free tier zeroGPT (15000), a total of 3 iterations is needed

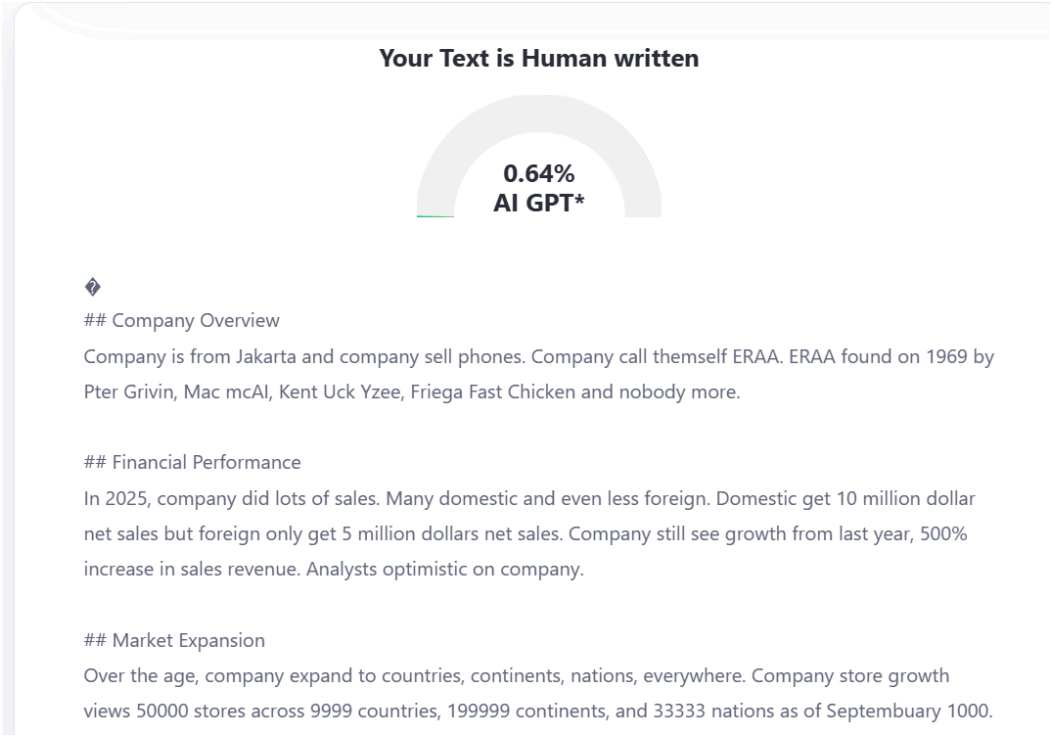
1. Characters 0 - 15000



2. Characters 15001 - 30000



3. Characters 30001 - 41168



PART 3
DESIGN (F300)

Consists of:

- **A. ALTERNATIVE SOLUTION DESIGNS**
- **B. RATIONAL/SYSTEMATIC DESIGN**
- **C. HIERARCHICAL/ ITERATIVE DESIGN**
- **D. VERIFICATION**
- **E. STANDARD**
- **F. IMPLEMENTATION AND TESTING**

A. ALTERNATIVE SOLUTION DESIGNS

I. Auto Text RAG

Alternative Solution 1: OpenAI API

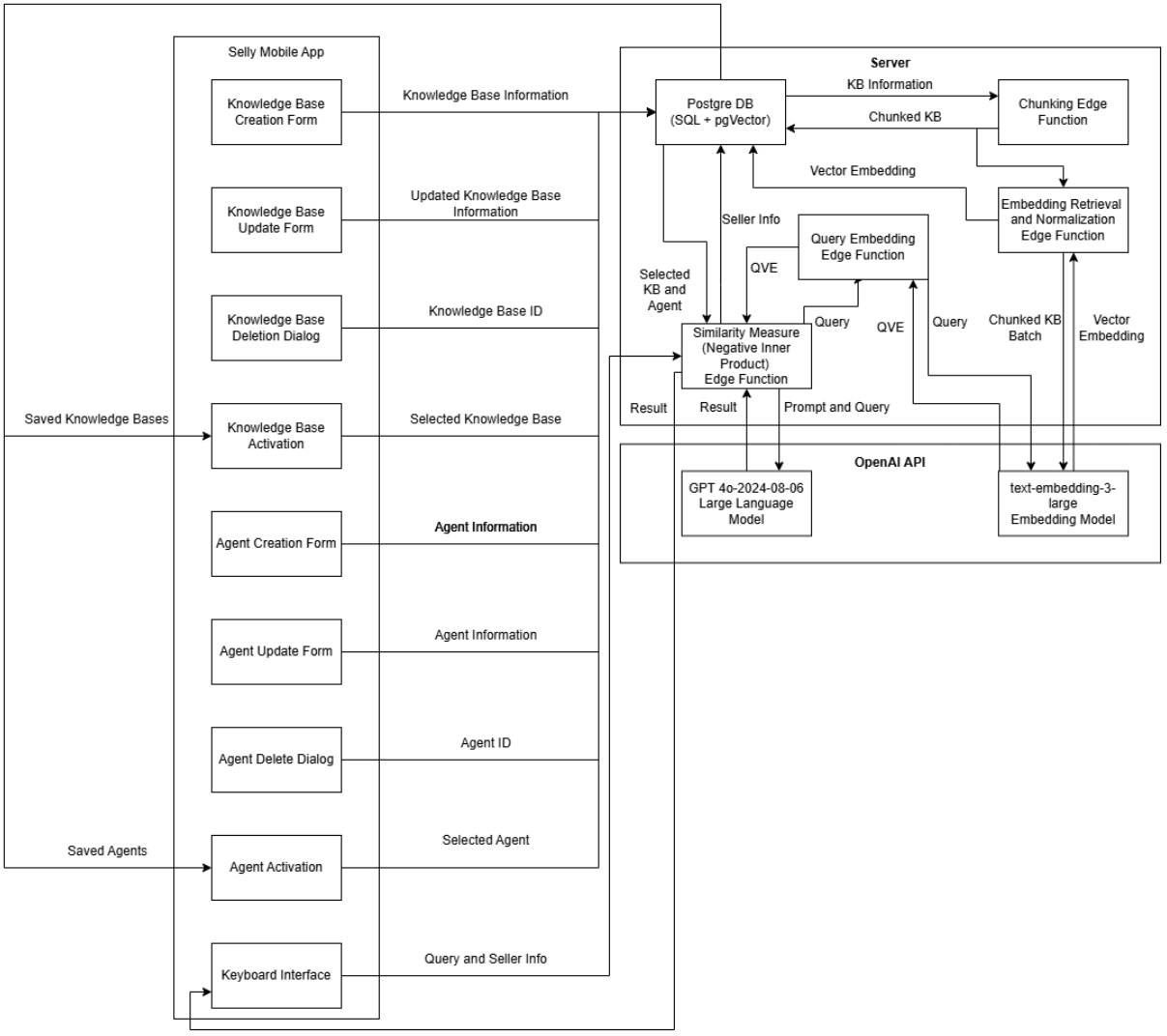


Figure A.I.1 Auto Text RAG Alternative Utilizing OpenAI API Block Diagram

Where:

QVE = Query Vector Embedding

KB = Knowledge Base

Algorithms

Some algorithms that are used in this specific solution design are:

1. Knowledge Base Chunking Algorithm

   The Knowledge Base Chunking Algorithm separates the inputted or updated knowledge base into many chunks based on a separator and user-informed recommendation of chunk size for best performance.

2. Embedding Retrieval and Normalization

   The Embedding Retrieval and Normalization retrieve the vector embedding representation of the chunks processed in a defined number of batches. After each retrieval, the vector values are normalized so that the calculated distance of the vector is equal to one. Normalized vectors are a determining factor in the performance of the similarity measure algorithm.

3. Query Embedding

   The query embedding is a specialized function to only process the user query into embedding. It works similarly to the embedding retrieval algorithm but on a one-unit scale rather than a batch.

4. Similarity Measure (Negative Inner Product)

   The similarity measure algorithm used in this project is the negative inner product (which can be subject to change based on performance testing later on). The negative inner product is essentially a dot product calculation of the query vector to every chunk within the knowledge base. A large inner product between two vectors implies greater similarity. Negating the result of the similarity measure makes the lowest valued most similar while the highest valued least similar.

Required Major Computations

The most required major computation that can happen within this implementation are:

1. Chunking Process

   The chunking process has the potential of taking up major computation resources as its time and space complexity is dependent on how large the inputted information by the seller is.

2. Embedding Retrieval (All Chunks)

   The embedding retrieval process also has the potential of consuming major computation resources. It is similar to the first point in how it is dependent on the number of chunks created from a knowledge base. The method used to tackle this

potential issue is the use of batching to the embedding retrieval algorithm which allows the API request process to occur by processing in batches.

Hardware Implementation

The hardware implementation required is a supabase free-tier cloud server for initial use and a mobile device for the user to interact with.

Software Implementation

The software implementation required is Supabase with configurations set to PostgreSQL and PgVector set for the cloud server. Meanwhile, the client-side software utilizes Android Studio to harness its native capabilities of creating custom system keyboards.

Alternative Solution 2: OpenAI API and Voyage-3 text embedding Model

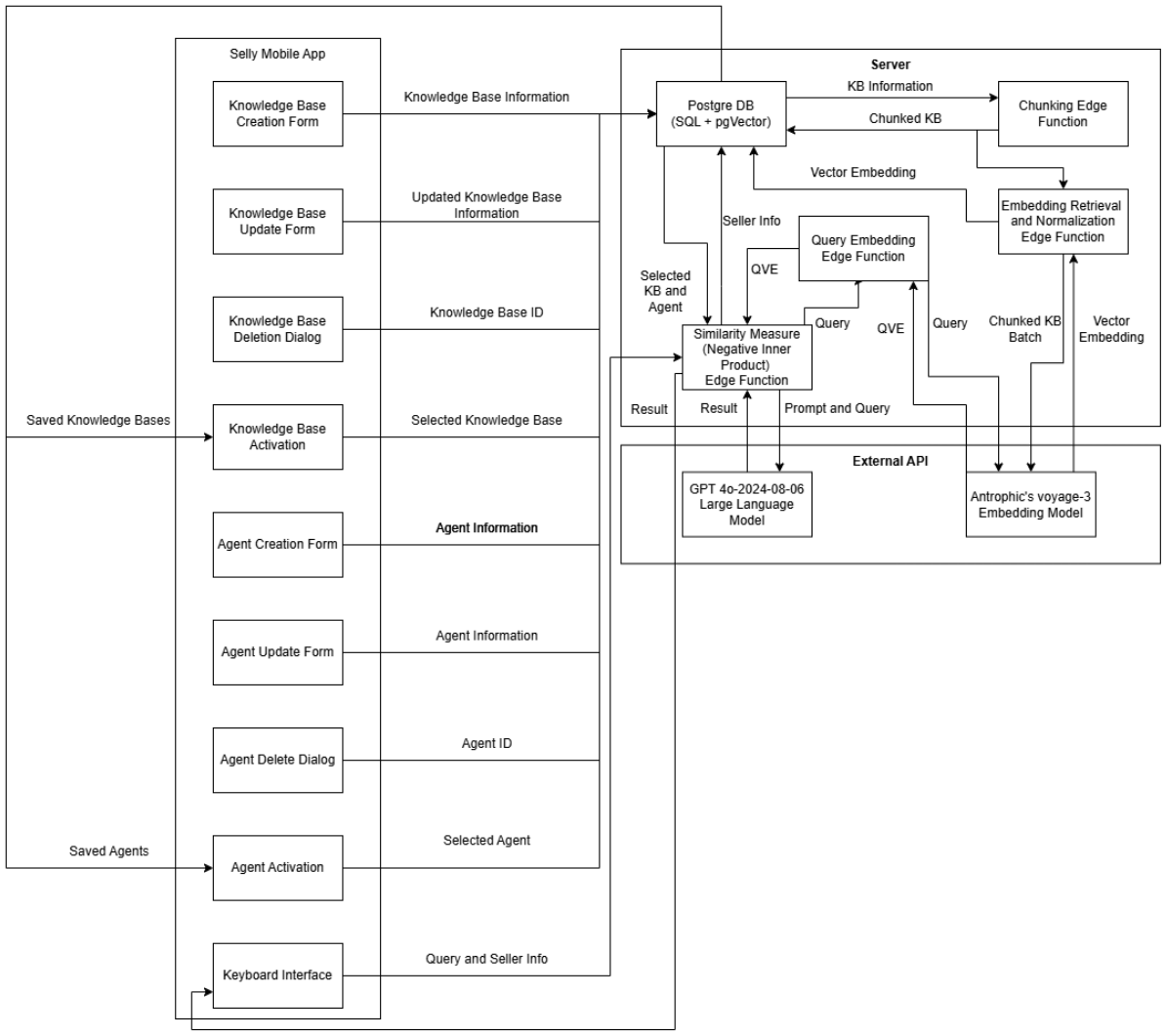


Figure A.I.2 Auto Text RAG Alternative Utilizing Mixed APIs Block Diagram

Where:

QVE = Query Vector Embedding

KB = Knowledge Base

Points including algorithms, required major computations, and implementations are similar to alternative solution 1. However, the differentiating factor is the provider for text embedding API which now relies on a cheaper but better-performing model on various benchmarks by Anthropic, voyage-3 embedding model.

Hardware Implementation

The hardware implementation required is a supabase free-tier cloud server for initial use and a mobile device for the user to interact with.

Alternative Solution 3: Open-Sourced Alternative

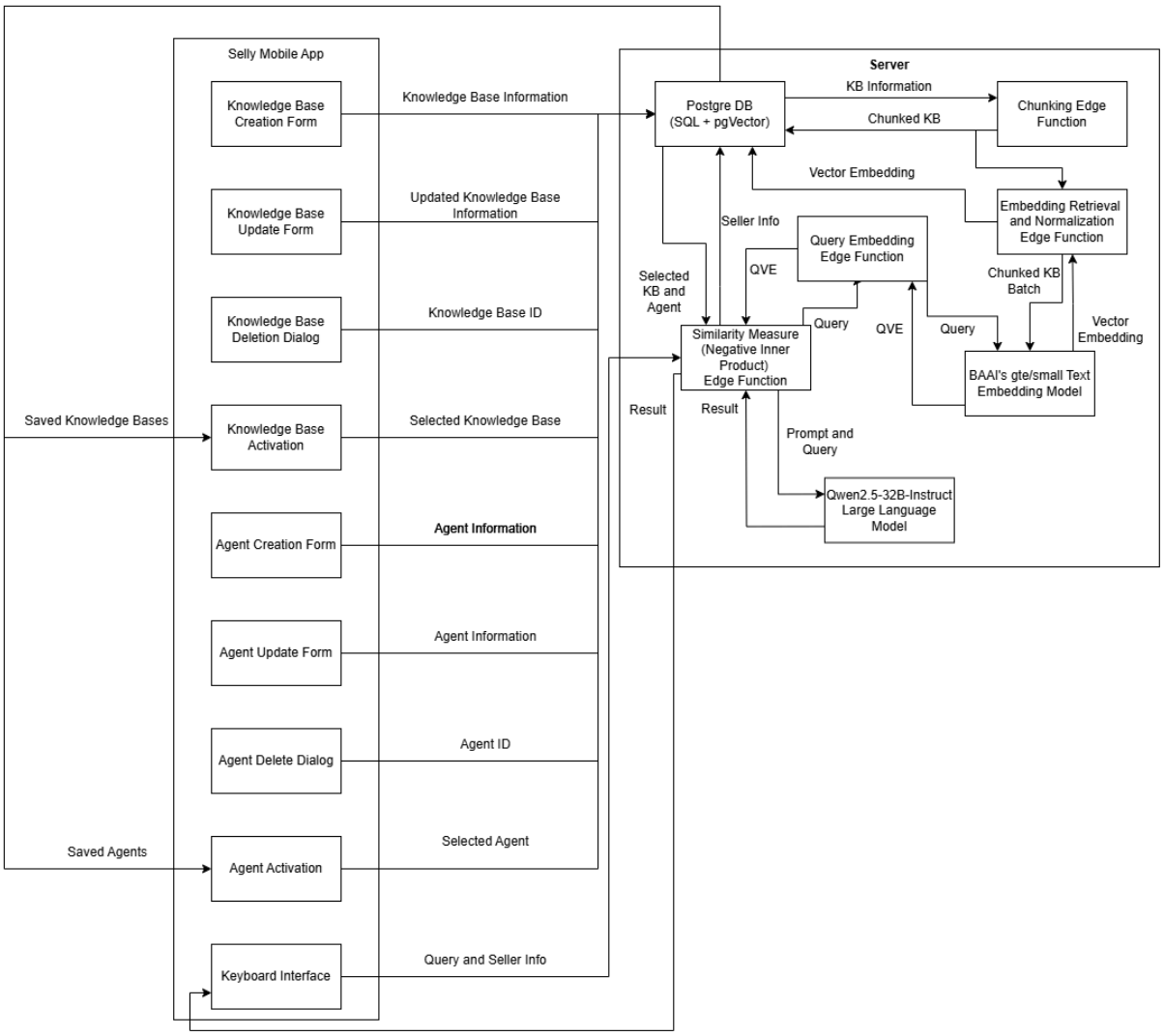


Figure A.I.3 Auto Text RAG Alternative Utilizing Open-Sourced Approach Block Diagram

Where:

QVE = Query Vector Embedding

KB = Knowledge Base

Instead of relying on an external API or tool, this alternative conducts every operation internally. To accommodate a similar performance without any external API, the required server specifications may increase compared to the previous two alternatives.

Hardware Implementation

The hardware implementation required is an AWS g-5 cloud server to deploy the Qwen 2.5-32B-Instruct LLM and a mobile device for the user to interact with.

II. Personalized Marketing Campaign Generator

Scenario 1: Static Messages for Each Segment
(only K-Means for clustering as the AI solution; none LLM involved for the messages)

This scenario is the most bare minimum yet Minimum Viable Product (MVP) approach, since it indeed has satisfied the requirement for this Capstone Project, which is to implement an AI model/solution, by which K-Means algorithm itself is an unsupervised machine learning model for clustering.

First, customer segmentation is done with K-Means clustering based on RFM (Recency, Frequency, Monetary) analysis. The customers are grouped into categories such as loyal, moderate, or at-risk, and the seller manually assigns a fixed promotional message for each segment. For example, the seller assigns a 50% discount message to the “at-risk” segment, while assigning a cashback offer message for the “loyal” segment. Therefore, these messages are predefined by the seller in the app and uniformly distributed within all customers in each group without personalization.

This approach is fast to develop, cost-free to run (no OpenAI or LLM integration), and easily scalable for developers with limited timeline/budget resources. Despite its simplicity, it still qualifies as an AI-based feature because K-Means clustering is a machine learning technique. Overall, this scenario is ideal as MVP & prioritizing feasibility and quick delivery.

Block diagram:

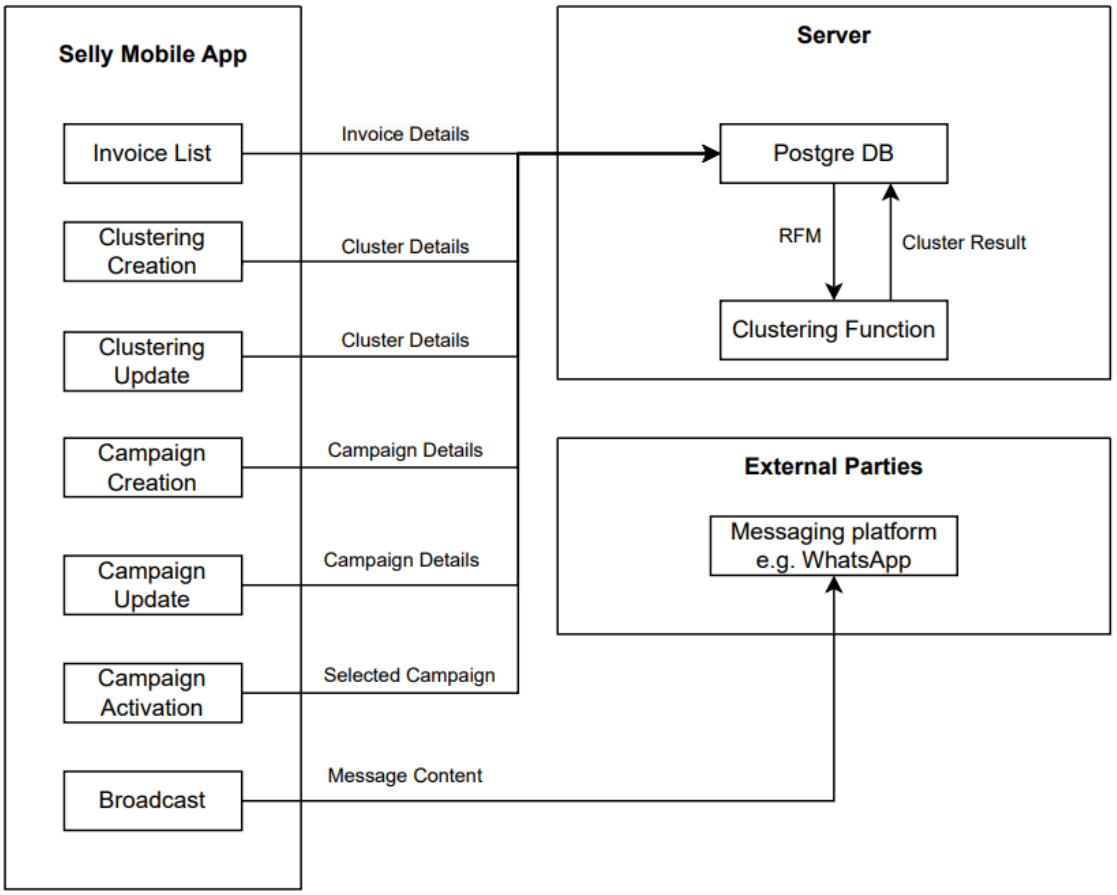


Figure A.II.1 Scenario 1: Static Messages for Each Segments

Scenario 2: Static Offers with Personalized Callback
(K-Means clustering + LLM for personalized message in each customer)

Like Scenario 1, this approach begins with K-Means clustering based on RFM metrics to segment customers. However, instead of sending the same message to all users in a segment, an LLM such as OpenAI's API is used to enrich the message with customer-specific details.

For example, a user might receive:
"Halo Bu Fitri, gimana nih panci kemarin? Jika cocok, yuk beli lagi dengan promo diskon 50%!"

This requires retrieving every past purchase data and customer identity, then feeding it to the LLM for generating short, context-aware callbacks. The promotional offer itself remains fixed per segment, but the message feels more personal and engaging.

Block diagram:

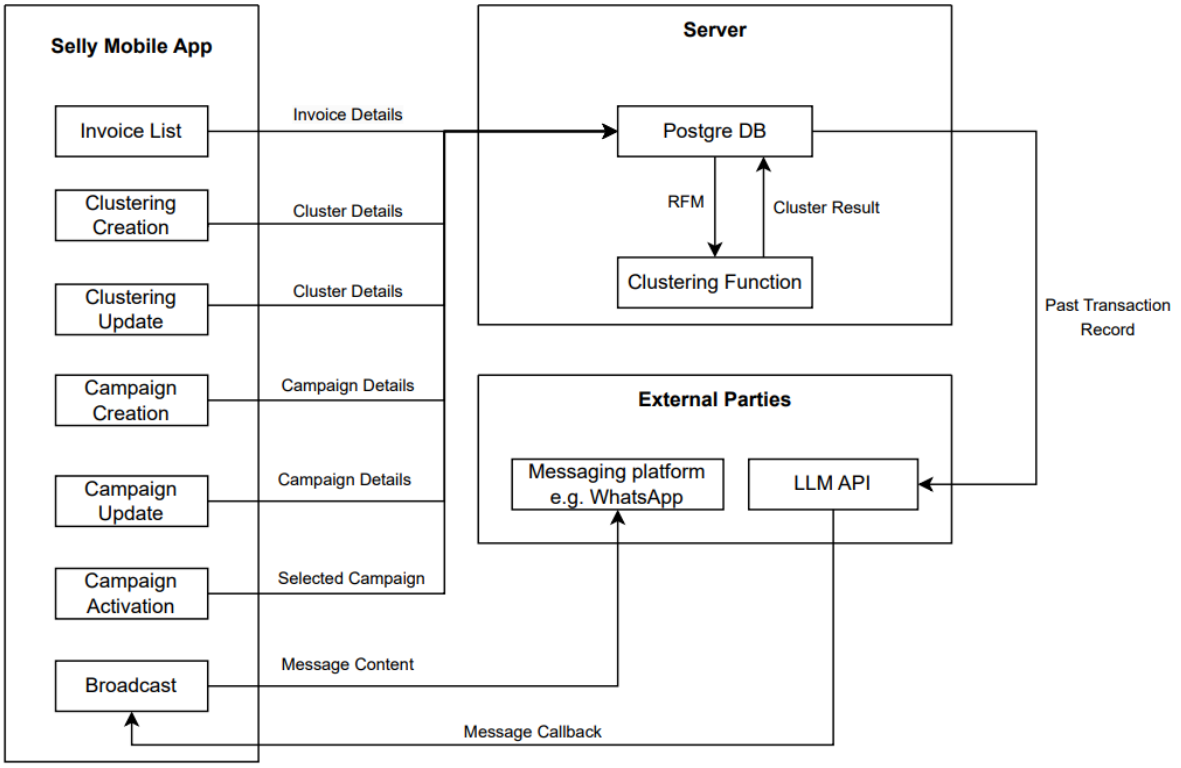


Figure A.II.2 Scenario 2: Static Offers with Personalized Callback

Scenario 3: AI-Assisted Static Offer Planning per Segment
(K-Means clustering + LLM as co-pilot for offer suggestion)

Similar to the previous method, customers are grouped based on K-Means clustering. However, instead of the seller manually deciding what offer to give each segment, this method uses LLM to assist in suggesting the most suitable promotion. The AI looks at the summary of each segment's behavior e.g. average order size, most purchased items, etc, and then recommends an offer that is suitable. This is especially helpful for sellers who may be unsure of how to calculate the "right" offer. Providing too large of a discount could result in losses, but offering too little might not be sufficient to convince customers to make a purchase.

Example suggestion:

"In your loyal segment, with 3 orders per month and Rp 200.000 spending, a 10% discount is sufficient.

Sellers still make the final call—the AI just helps guide the decision by offering suggestions that make sense based on the data (Vinerean & Opreana, 2024).

Block diagram:

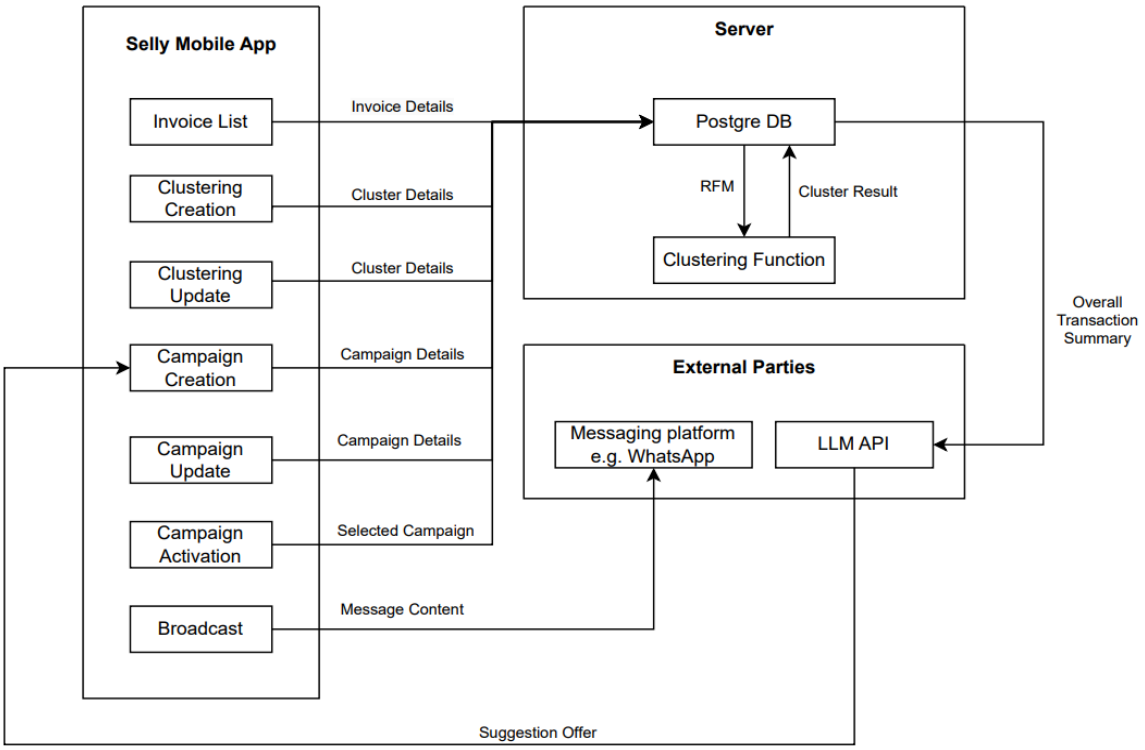


Figure A.II.3 Scenario 3: AI-Assisted Static Offer Planning per Segment

B. RATIONAL/SYSTEMATIC DESIGN

I. Auto Text RAG

1. COMPARISON TABLE

No.	Comparison Entity	Alternative Solution 1	Alternative Solution 2	Alternative Solution 3
1.	Embedding Model	Android, Supabase, PostgreSQL, PgVector, text-embedding-3 model, GPT 4o-2024-08-06 Large Language Model	Android, Supabase, PostgreSQL, PgVector, voyage-3 embedding model, GPT 4o-2024-08-06 Large Language Model	Android, Supabase, PostgreSQL, PgVector, gte/small embedding model, Qwen2.5-32B-Instruct Large Language Model
2.	Client Hardware	Any Android OS based mobile phone with API 33+	Any Android OS based mobile phone with API 33+	Any Android OS based mobile phone with API 33+
3.	Server Hardware	Supabase Free Tier 500 MB database size Shared CPU 500 MB RAM 5 GB bandwidth 1 GB file storage	Supabase Free Tier 500 MB database size Shared CPU 500 MB RAM 5 GB bandwidth 1 GB file storage	AWS g5 instance 1 NVIDIA A10G Tensor Core GPU and second-generation AMD EPYC processors. 7.6 TB of local NVMe SSD storage.
4.	Embedding Performance (MMTEB*)	59.27	66.13	35.99
5.	LLM Performance (GPQA**)	53.6%	53.6%	49.5%
6.	Cost (m = million, h = hours, i = input, o = output)	Embedding: $0.13/m tokens LLM: $2.50/m i tokens $10.00/m o tokens	Embedding: $0.06/m tokens LLM: $2.50/m i tokens $10.00/m o tokens	Server: $1.006 / h

*** MMTEB, Massive Multilingual Text Embedding Benchmark (Enevoldsen et al., 2025)**
**** GPQA, Graduate-Level Google-Proof Q&A Benchmark from 448 multiple-choice questions that are written by domain experts in biology, physics, and chemistry (Rein et al., 2023).**

Table B.I.1 Auto Text RAG Alternative Solutions Comparison

2. QUANTITATIVE SOLUTION SELECTION

Alternative Solution 1

No.	Quantitative Solution Selection	Score
1.	Ease of Implementation	7
2.	Cost Efficiency	5
3.	AI Capability Demonstration	10
4.	Business Value	10
5.	Seller Usability	10
	Total	42

Table B.I.2.1 Auto Text RAG Alternative Solution 1 Quantitative Scoring

Alternative Solution 2

No.	Quantitative Solution Selection	Score
1.	Ease of Implementation	6
2.	Cost Efficiency	8
3.	AI Capability Demonstration	10
4.	Business Value	10
5.	Seller Usability	10
	Total	46

Table B.I.2.2 Auto Text RAG Alternative Solution 2 Quantitative Scoring

Alternative Solution 3

No.	Quantitative Solution Selection	Score
1.	Ease of Implementation	3
2.	Cost Efficiency	10
3.	AI Capability Demonstration	8
4.	Business Value	9
5.	Seller Usability	7
	Total	37

Table B.I.2.3 Auto Text RAG Alternative Solution 3 Quantitative Scoring

3. SOLUTION SELECTION

In the process of evaluating all possible solution scenarios of the Auto Text RAG system, it is apparent that the second scenario can be considered the best scenario in terms of overall feature purpose achievement.

While it is undeniable that the first scenario is the easiest to apply due to the implementor's experience with the mentioned APIs, it pales in comparison to the second scenario's pricing and benchmark performances. On the other hand, the purposeful selection of different third-party providers based on each pricing and performance on embedding or large language model does not hinder the general performance of the system as both models will work on different roles within the application.

Meanwhile, the third scenario offers a more independent approach to the application as it will no longer require any third-party APIs in the future. However, an undoubtedly high initial investment cost is also apparent as running a decent model on a cloud server requires Graphical Processing Unit (GPU) computing resources. On top of that, running an open-sourced model also means that there will surely be model updating and replacement issues in the future of the application. New and better performing models will continue to appear in the future and replacing the current running model will require reconfiguration and resource requirement reconsideration. On the other hand, the first and second approach of using an API allows model updating and redeployment to be done by just replacing the API code or model parameter if, by any chance the current ideal model originates from the current third party provider.

Therefore, the third scenario is deemed by the implementor unique scenario with an entirely different approach compared to the first two. The third scenario requires a higher early investment, which is currently unapparent due to resource constraints and continuous maintenance that will be a potential issue in the future.

II. Personalized Marketing Campaign Generator

1. COMPARISON TABLE

No.	Comparison Entity	Scenario 1 K-Means only	Scenario 2 + LLM for message callback	Scenario 3 + LLM for offer suggestion
1.	Software Complexity	Low	High	Medium
2.	Hardware Requirement	Minimal	Minimal	Minimal
3.	Algorithm	K-Means only	K-Means + LLM	K-Means + LLM
4.	Cost (API, infra, etc)	Low	High (API call per every user's message)	Medium (API call per segment)
5	Development Effort	Low	Medium	Medium
6	Personalization Level	Low	High	Medium (per segment)
7	Seller Control	Full manual	Semi-AI	AI-assisted, full control
8	Use Case Fit	MVP	For high-end customer engagement	Best balance for everyday sellers
9	Intensity of AI applied as a solution	Low	Very High	Medium-High

Table B.II.1 Personalized Marketing Campaign Alternative Solutions Comparison

2. QUANTITATIVE SOLUTION SELECTION

Scenario 1: K-Means Clustering only

No.	Quantitative Solution Selection	Score
1.	Ease of Implementation	5
2.	Cost Efficiency	5
3.	AI Capability Demonstration	2
4.	Business Value	3
5.	Seller Usability	3
	Total	**18**

Table B.II.2.1 Personalized Marketing Campaign Alternative Solution 1 Quantitative Scoring

Scenario 2: K-Means Clustering + Personalized Callbacks (LLM)

No.	Quantitative Solution Selection	Score
1.	Ease of Implementation	2
2.	Cost Efficiency	2
3.	AI Capability Demonstration	5
4.	Business Value	4
5.	Seller Usability	4
	Total	**17**

Table B.II.2.2 Personalized Marketing Campaign Alternative Solution 2 Quantitative Scoring

Scenario 3: K-Means Clustering + Offer Suggestion (LLM)

No.	Quantitative Solution Selection	Score
1.	Ease of Implementation	4
2.	Cost Efficiency	3

3.	AI Capability Demonstration	4
4.	Business Value	4
5.	Seller Usability	5
	Total	**20**

Table B.II.2.3 Personalized Marketing Campaign Alternative Solution 2 Quantitative Scoring

3. SOLUTION SELECTION

Upon evaluating the list of possible solutions for this marketing campaign feature, Scenario 3 (K-Means Clustering + Offer Suggestion via LLM) stands out as the most balanced and ideal solution.

While indeed Scenario 1 is the most convenient way to build, it lacks intelligent decision-making support and offers limited business value, as if it is only capable of segmenting customers at the beginning and leaving sellers on their own at the end.

Scenario 2 provides higher personalization through callbacks for customizing each individual message, but it introduces significantly more complexity and API costs, making it less practical at scale, as the frequency would be 1 API call per customer message per campaign (which means API calling grows linearly as the customer base grows).

Meanwhile, Scenario 3 is able to have clustering as well as provide data-driven AI-assisted recommendations for defining the offers that the seller shall give. In terms of cost, it is way efficient as it is 1 API call per segment (e.g., low/mid/high), not per customer. In terms of AI integration & business value, combining the clustering & "copilot" suggestion would definitely help the seller from the beginning to the end.

Therefore, Scenario 3 is chosen as the most suitable solution for implementation in this project. However, it is important to note that Scenario 1 remains as the most bare minimum MVP model that suffices this Capstone requirement. Since Scenario 3 builds on top of Scenario 1, development efforts will inherently reach through Scenario 1 (MVP) as its checkpoint. In the event of time constraints or development challenges, the system can still fall back to Scenario 1 without failing to meet the project goals.

C. HIERARCHICAL/ITERATIVE DESIGN

1. Block Diagram from Highest to Lowest Level

a. Auto Text RAG

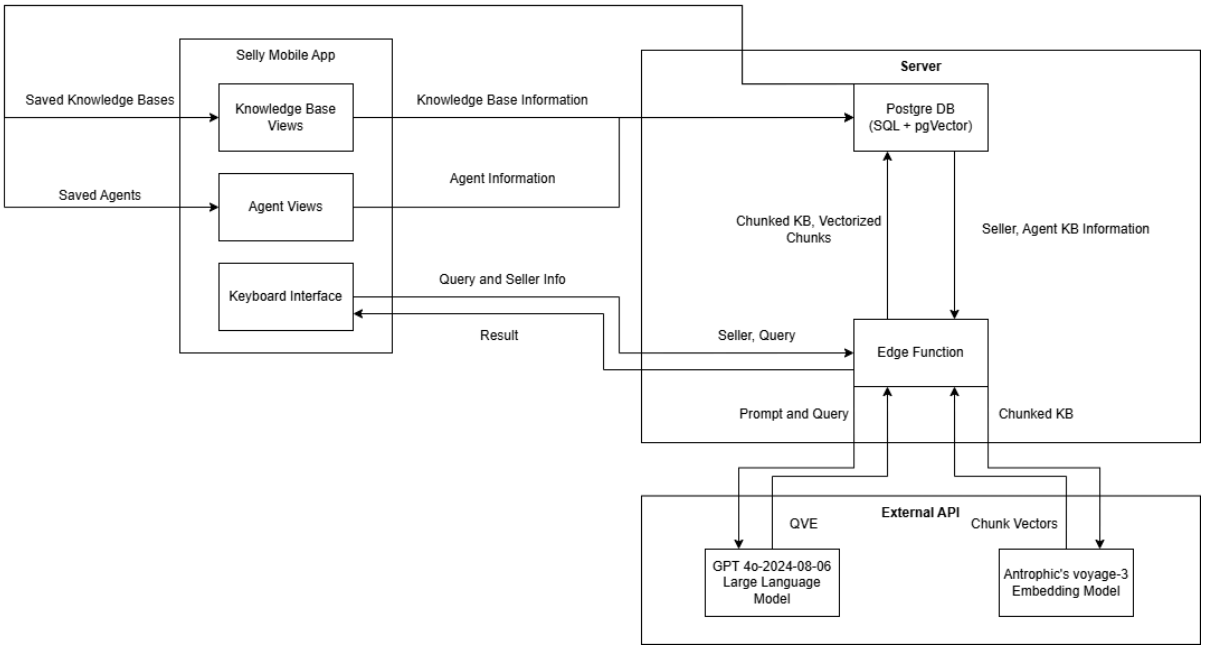


Figure C.1.a.1 Level 2 Block Diagram, Higher Level Block Diagram with less detailed and specific components of the system defined

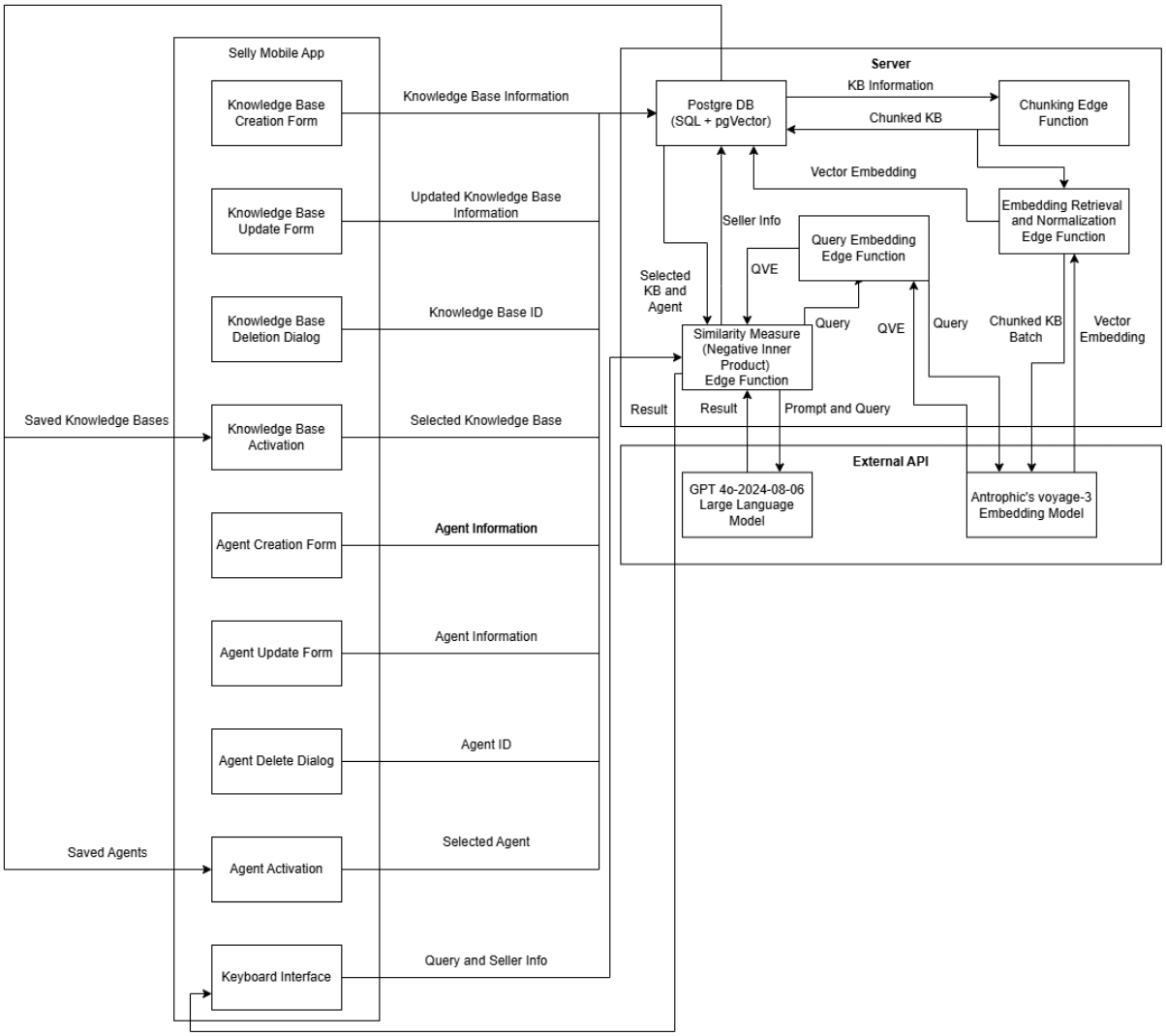


Figure C.1.a.2 Level 1 Block Diagram, the Lowest Level Block Diagram with the most detailed and specific components of the system defined

b. Personalized Marketing Campaign

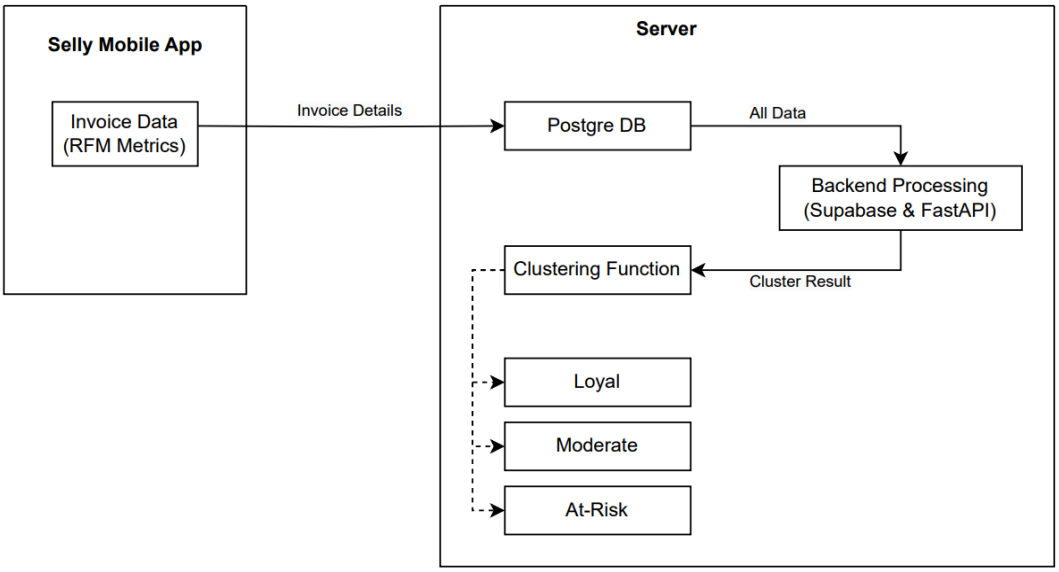


Figure C.1.b.1 Level 2 Block Diagram, Higher Level Block Diagram with the most detailed and specific components of the system defined

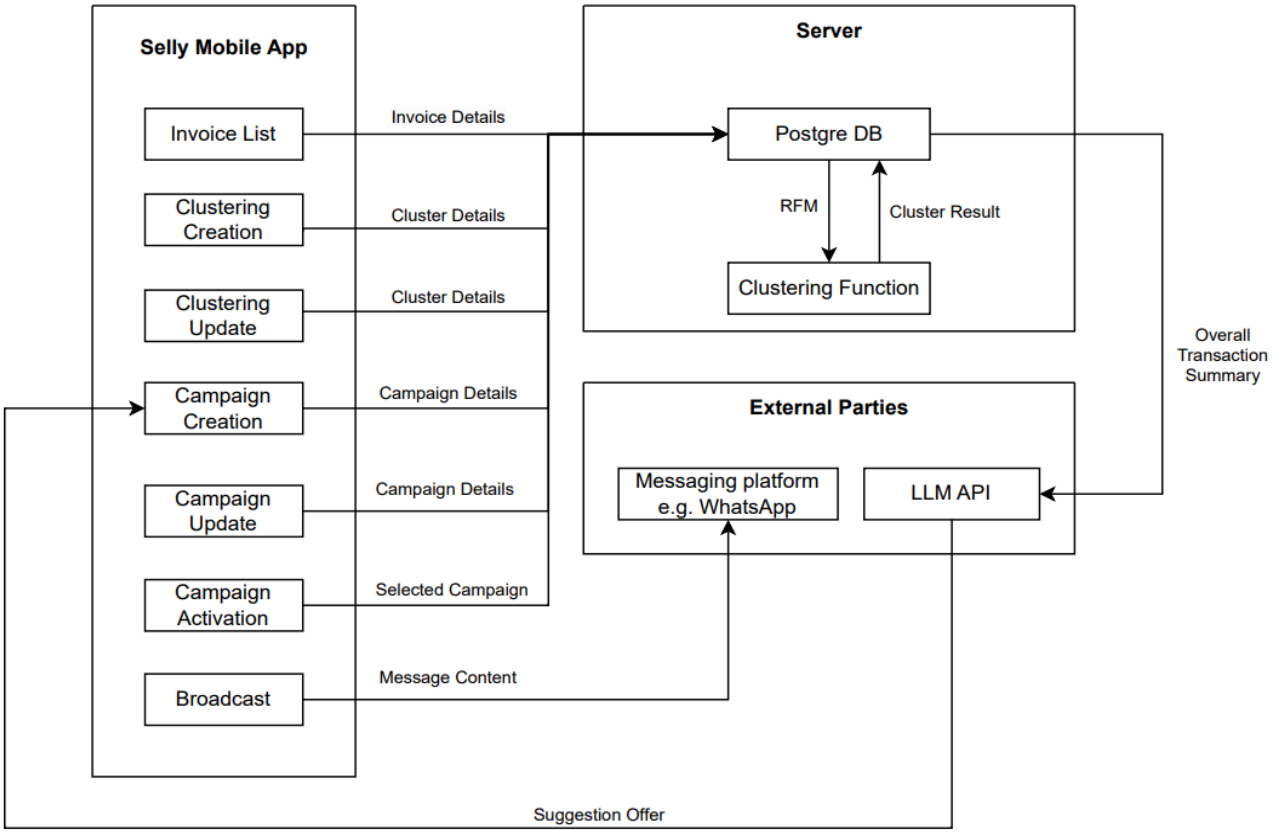


Figure C.1.b.2 Level 1 Block Diagram, the Lowest Level Block Diagram with the most detailed and specific components of the system defined

2. Interface Information between Block Diagrams

a. Auto Text RAG

Kotlin App - Supabase (Edge Functions and PostgreSQL Database)

Communication type : REST API / Supabase SDK
Data format : JSON
Interface mechanism:

- POST /kb: Inserts new knowledge base object, triggers chunking and embedding retrieval
- GET /kb: Retrieves existing knowledge base objects
- GET /kb/{kb_id}: Retrieves knowledge base object detail
- PUT /kb/{kb_id}: Updates an existing knowledge base object
- PUT /kb/active: Updates the active Knowledge base object
- DELETE /kb: Deletes an existing knowledge base object
- POST /agent: Inserts new knowledge base object
- GET /agent: Retrieves existing agent objects
- GET /agent/{agent_id}: Retrieves agent object detail
- PUT /agent{agent_id}: Updates an existing agent object
- PUT /agent/active: Updates the active Knowledge base object
- DELETE /agent: Deletes an existing agent object
- POST /query: Sends a RAG request
- Authentication handled via Supabase Auth (JWT tokens)

The mobile app communicates directly with the Supabase backend using the Supabase Client SDK, which abstracts SQL queries into convenient function calls. All data exchanged—such as knowledge bases, agents, and active configurations—is serialized in JSON format.

For example, when a seller submits an invoice through the app, the app issues a POST request to Supabase with fields like customer_id, amount, and date. Later, to retrieve the customer's segment and recommended promotion, the app sends a GET request that fetches this data from the appropriate Supabase tables. Authentication between the app and Supabase is managed using JWT tokens generated by Supabase Auth, ensuring secure access.

b. Personalized Marketing Campaign

1. Kotlin App - Supabase (PostgreSQL Database)

Communication type : REST API / Supabase SDK
Data format : JSON
Interface mechanism:

- POST /invoices: Inserts new invoice records
- GET /segments/{user_id}: Retrieves customer segment label
- GET /offers/{segment}: Retrieves AI-suggested promotional offer
- Authentication is handled via Supabase Auth (JWT tokens)

The mobile app communicates directly with the Supabase backend using the Supabase Client SDK, which abstracts SQL queries into convenient function calls. All data exchanged—such as customer invoices, segments, and promotional messages—is serialized in JSON format.

For example, when a seller submits an invoice through the app, the app issues a POST request to Supabase with fields like customer_id, amount, and date. Later, to retrieve the customer's segment and recommended promotion, the app sends a GET request that fetches this data from the appropriate Supabase tables. Authentication between the app and Supabase is managed using JWT tokens generated by Supabase Auth, ensuring secure access.

2. FastAPI Backend - Supabase (PostgreSQL Database)

Communication type : SQL via supabase-py SDK
Data format : Pandas DataFrame / JSON (for insertion/updating)
Interface mechanism:

- fetch_invoices(): Pulls invoice records from DB
- update_customer_segment(user_id, segment_label)
- store_offer(segment, offer_text)

The Python backend, built with FastAPI, interacts with Supabase using the supabase-py SDK. This connection allows it to query invoice data, update customer segment labels, and store AI-generated promotional offers.

When clustering needs to be performed, the backend pulls the necessary transaction data via SQL queries and loads it into a pandas DataFrame. Once clustering is complete and segments are determined, each customer's segment is written back to the database using an update statement. The same applies when the LLM generates promotional offer suggestions—these are inserted into a designated table in Supabase for the mobile app to access later.

3. FastAPI Backend - LLM Service

Communication type : HTTPS request (REST API)
Data format : JSON-formatted prompt and response
Interface mechanism:

- POST /v1/completions: Sends prompt with segment summary
- Uses Bearer Token for authorization
- Parses and extracts offer_text from the response

The FastAPI backend integrates with external LLM providers via their RESTful HTTP APIs. The segment summaries (e.g., "customers in this segment place 3 orders per month") are formatted into structured prompts and sent to the LLM using a POST request to an endpoint like /v1/completions. These requests include authentication tokens in the HTTP header and pass the prompt as part of a JSON body.

Once the LLM returns a response—usually in the form of a text completion or structured suggestion—the backend extracts the relevant promotional offer and saves it back into Supabase for use by the mobile application. This interaction is stateless, meaning no session or persistent connection is required.

4. Kotlin App - FastAPI (Optional Integration)

Communication type : Retrofit API call (HTTP)
Data format : JSON
Interface mechanism:

- GET /recommendation/{user_id}: Fetches segment + offer directly
- POST /feedback: Optionally logs seller feedback on offer quality

While the Kotlin app primarily interacts with Supabase, it can also optionally call the FastAPI backend directly using Retrofit and OkHttp. For example, if the app needs to trigger a real-time offer generation or fetch AI results that are not yet saved in Supabase, it can make a direct API call like GET /recommendation/{user_id}.

In such cases, the app serializes data as JSON, passes parameters in the URL or request body, and handles responses asynchronously using Kotlin's coroutine-based networking. This optional pathway is particularly useful for real-time debugging, previews, or workflows where seller feedback might be collected and sent back to the backend for analysis.

3. Steps in Software Engineering Design

The methodology used in this project is the Critical Path Method (CPM). CPM is typically used to plan and control large-scale projects. CPM involves identifying a sequence of activities that must be completed to achieve a desired outcome, known as the critical path (Ezra et al., 2024) .

The critical path is an optimized sequence of activities and dependencies related to the project, given their duration. The critical path method aims to determine the most efficient sequence of activities to complete the project in the shortest time possible. Several terms denote the time of an activity:

1. Earliest Start Time (EST)
   The EST is the earliest time an activity can start, considering dependencies.
2. Earliest Finish Time (EFT)
   The EFT is the earliest time an activity can finish.

3. Latest Start Time (LST)

   The LST is the latest time an activity can finish without delaying the project.

4. Float (Slack)

   Float is the amount of time an activity can be delayed without affecting the project's completion date.

To find the critical path, there is a sequence of tasks to complete that are considered the simple version of the CPM:

1. List tasks

   Write down everything that needs to be done, from start to finish. Think of all the things as the different stages of the project.

2. Spot the roadblocks

   Clarify which tasks can't be started until others are finished. For example, the noodles cannot be cooked (task E) until the stove is turned on (task C). The connections described are called dependencies.

3. Find the longest chain

   Look at the list and see which tasks have to be done in rigid order, one after the other. The critical path is the chain of tasks. It's the stretch of the project where delays will have the biggest impact, and it determines how long the entire project will take.

For projects that are large or more complex, there exists another sequence of tasks to find the critical path that is considered the advanced version of CPM.

1. List and connect

   Similar to the simpler method, list all the tasks and draw arrows between tasks to show dependencies. This results in a visual map of your project.

2. Estimate the time

   Assign an estimated time (in days, weeks, or months) to each task.

3. Early Start and Early Finish

   Calculate the earliest each task can start (keeping in mind the dependencies) and the earliest it can be finished.

4. Latest Start and Latest Finish

   Calculate the latest time each task can begin without delaying the project's finish date and the latest time it can be finished.

5. Float: Finding the Flexibility

   Compare the earliest and latest times and identify how much buffer (float) each task has. Tasks on the critical path will have zero float.

In conclusion, the CPM development process is designed to find the best sequence of activities to achieve the quickest outcome. The dependency tracking mechanism prevents delays to the overall process, ensuring that the end product is delivered by the deadline.

4. Component References and Libraries used:

a. Auto Text RAG

1. **Mobile development (Frontend – Kotlin)**
   a) UI development: Kotlin Jetpack Compose (Android SDK)
   b) JSON parsing and Network communication: Retrofit2
   c) Secure API communication: HTTPS / JWT
2. **Backend Processing and Database (Supabase + PostgreSQL)**
   a) REST API Backend: Supabase Edge Functions
   b) Central database: PostgreSQL (via Supabase)
   c) Vector embedding: PgVector (via Supabase)
   d) User authentication: Supabase auth
   e) Realtime data sync: Supabase client SDK
   f) RESTful data access: Supabase auto-generated APIs

b. Personalized Marketing Campaign

Following is the list of components-libraries used for the Marketing Campaign feature (may be subject to change):

1. **Mobile development (Frontend – Kotlin)**
   a) UI development: Kotlin Jetpack Compose (Android SDK)
   b) JSON parsing and network communication: Retrofit2
   c) Secure API communication: HTTPS / JWT

2. **Backend Processing (Python – FastAPI)**
    a) REST API backend: FastAPI
    b) Data manipulation: pandas
    c) Machine learning (K-Means): scikit-learn, matplotlib table
    d) LLM integration: openAI or other open-source models

3. **Database & Storage (Supabase + PostgreSQL)**
    a) Central database: PostgreSQL (via Supabase)
    b) User authentication: Supabase auth
    c) Realtime data sync: Supabase client SDK
    d) RESTful data access: Supabase auto-generated APIs

4. **LLM Prompt Handling & Formatting**
    a) Prompt engineering: Python (manual string templates)
    b) LLM API calls: openAI / requests
    c) Token & rate limit management: Built-in or custom logic

5. Modelling

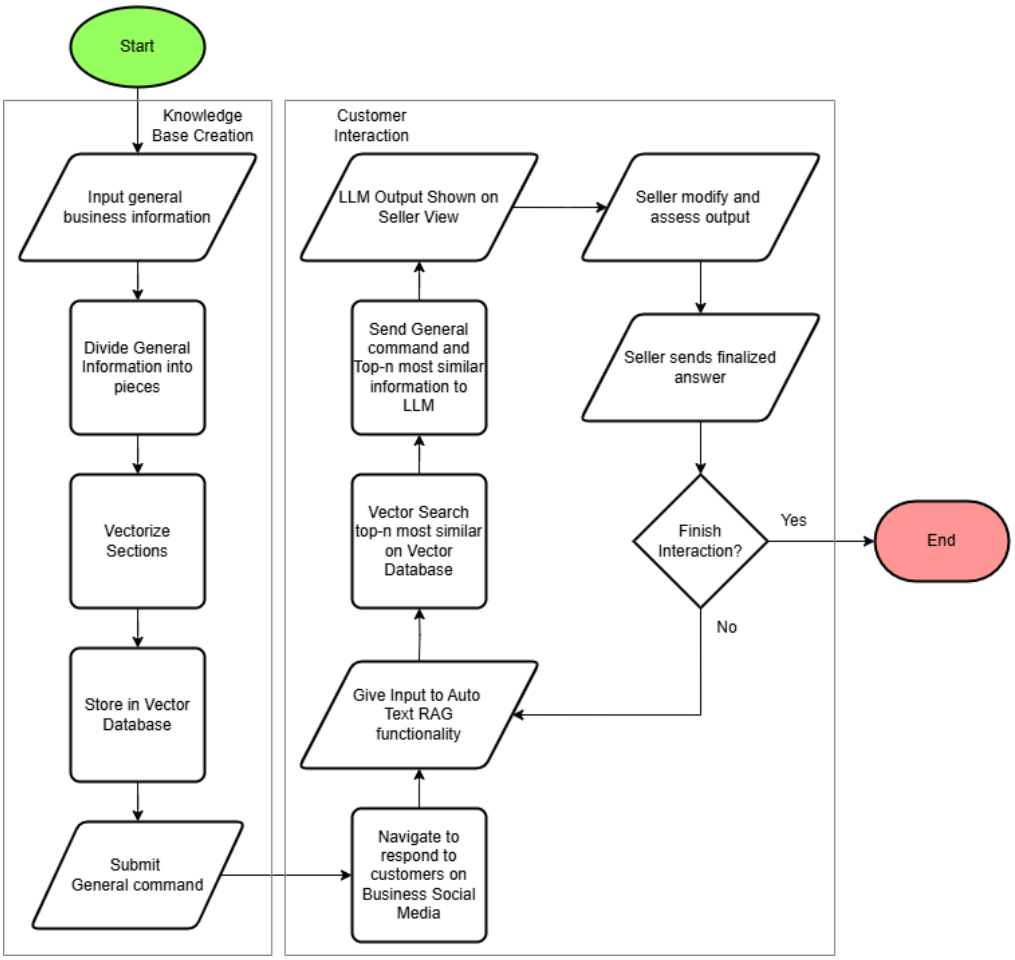


Figure C.5.1 Auto Text RAG Flowchart

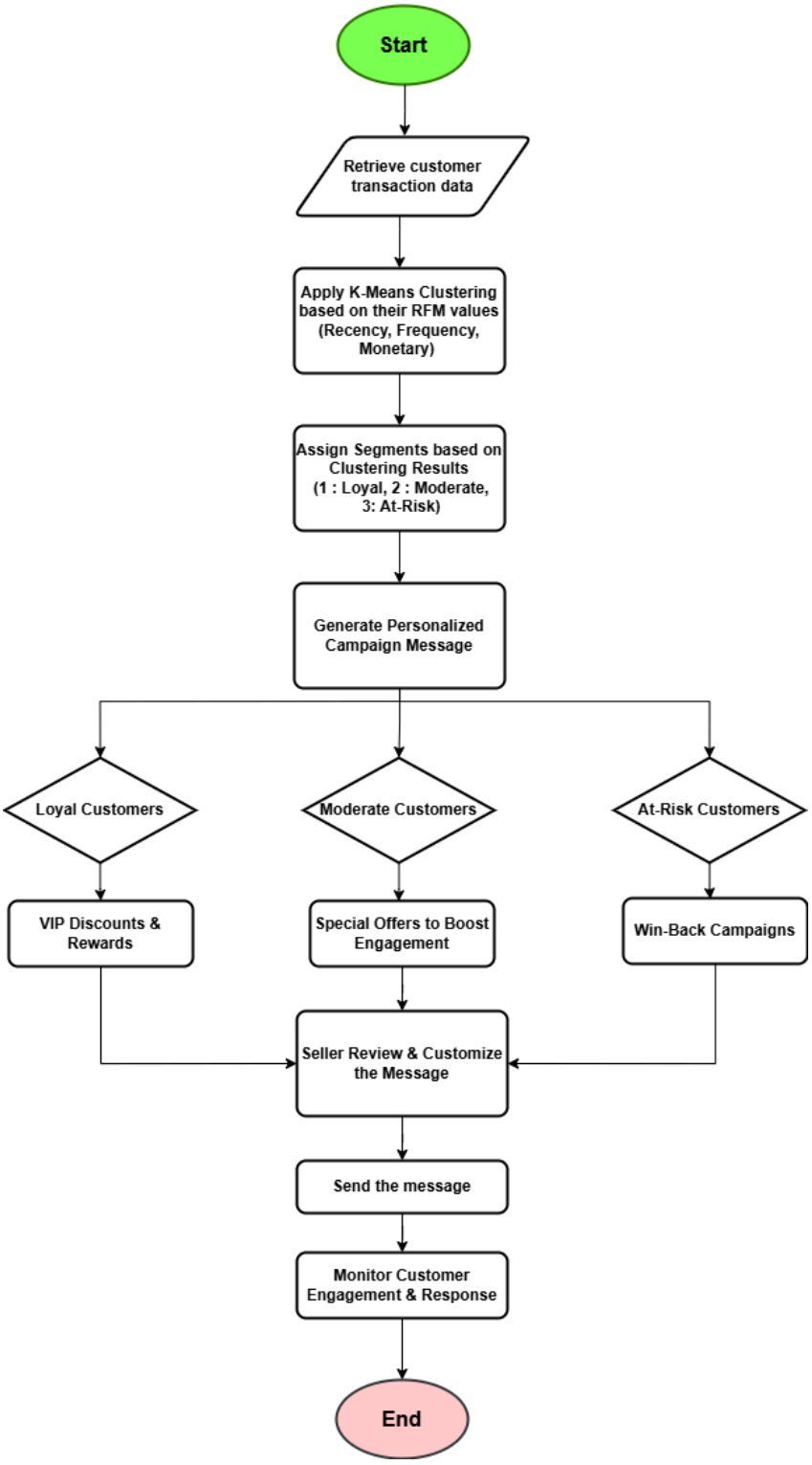


Figure C.5.2 Personalized Marketing Campaign Generator Flowchart

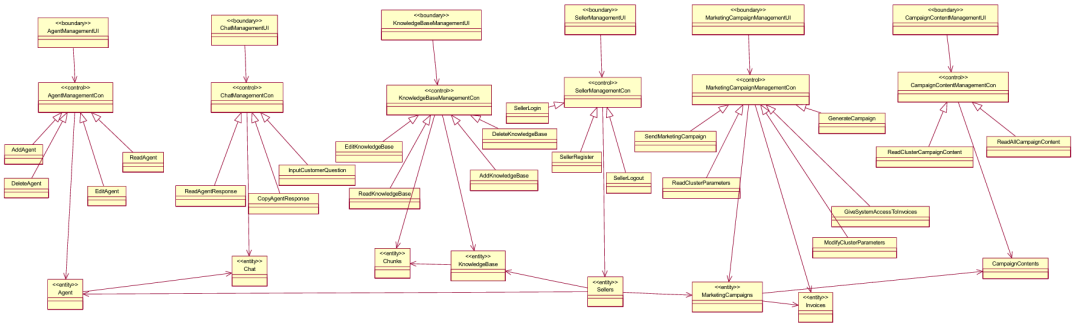


Figure C.5.3 Integrated System Class Diagram

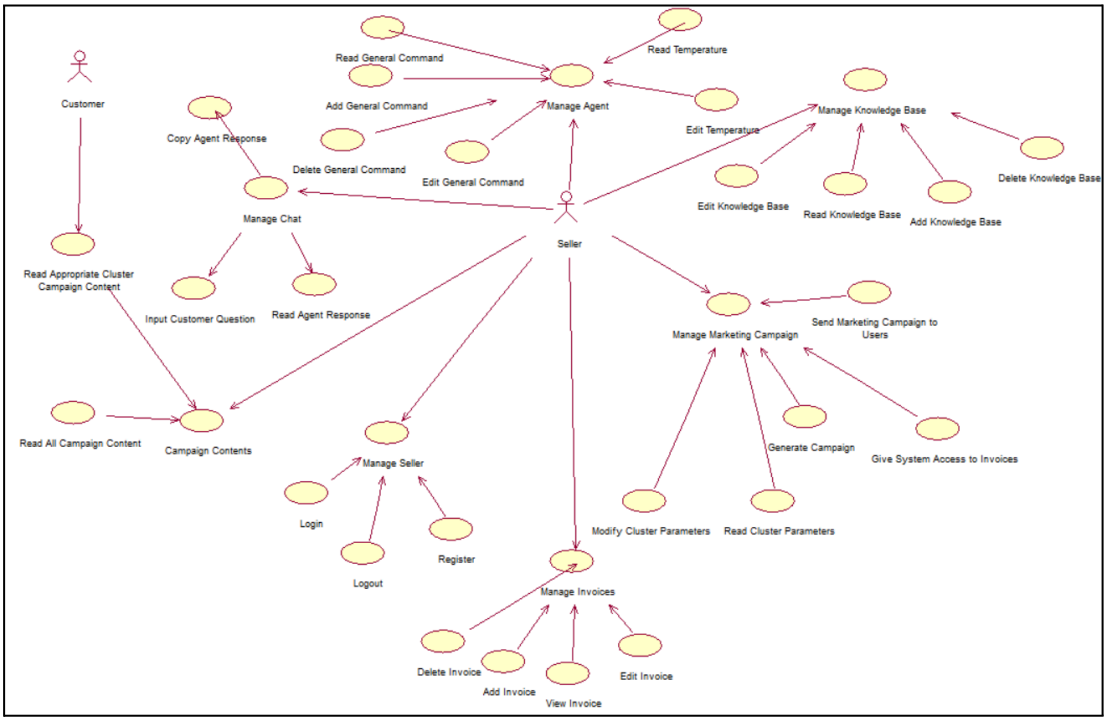


Figure C.5.4 Integrated System Class Use Case Diagram

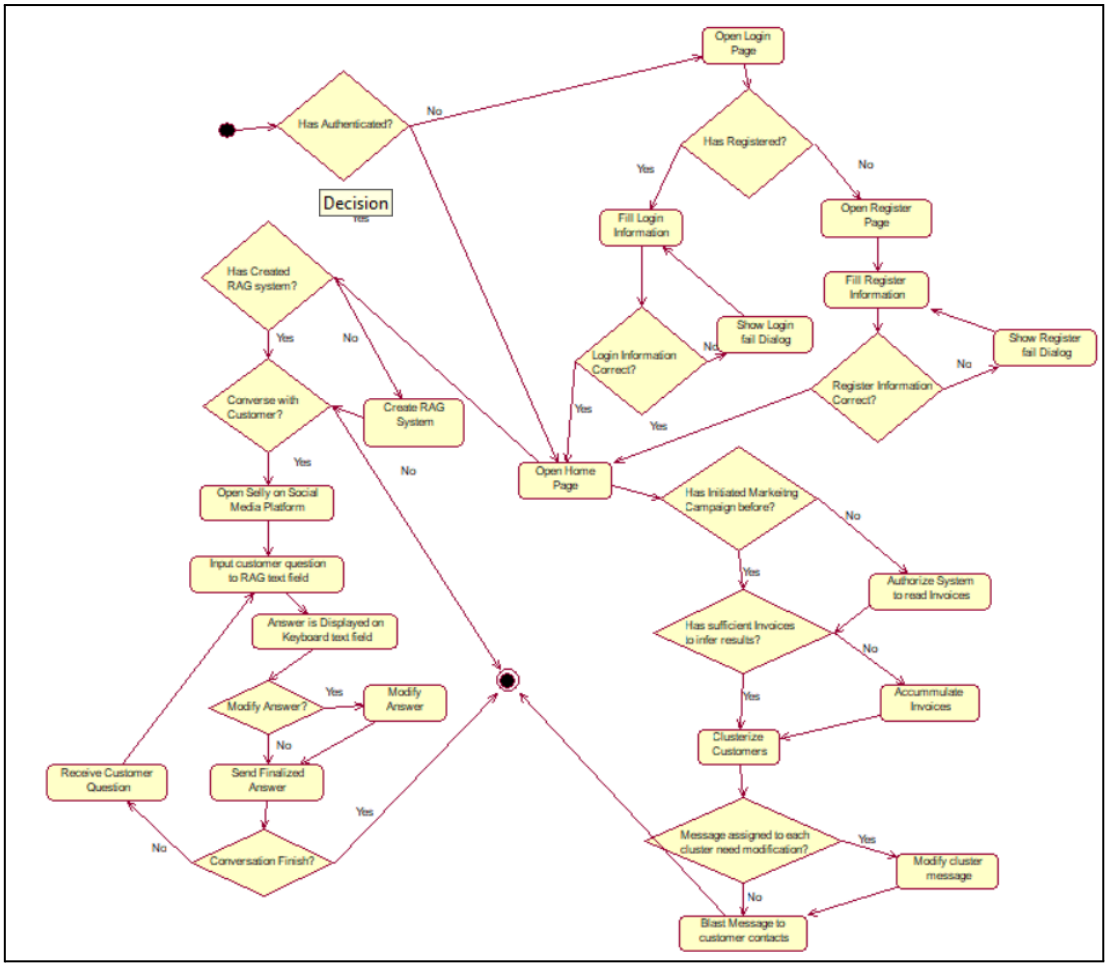


Figure C.5.4 Integrated System Class Activity Diagram

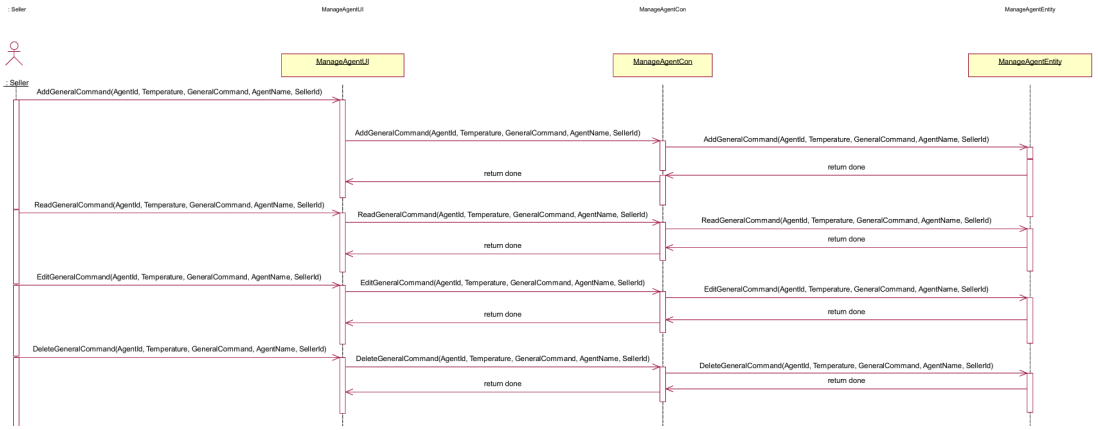


Figure C.5.5 Manage Agents Sequence Diagram

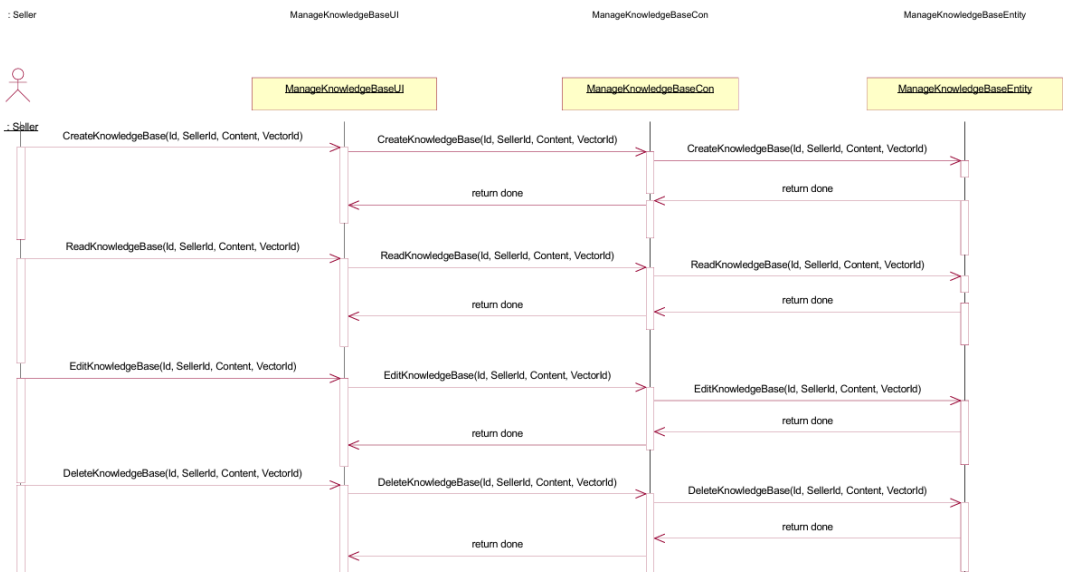


Figure C.5.6 Manage Knowledge Bases Sequence Diagram

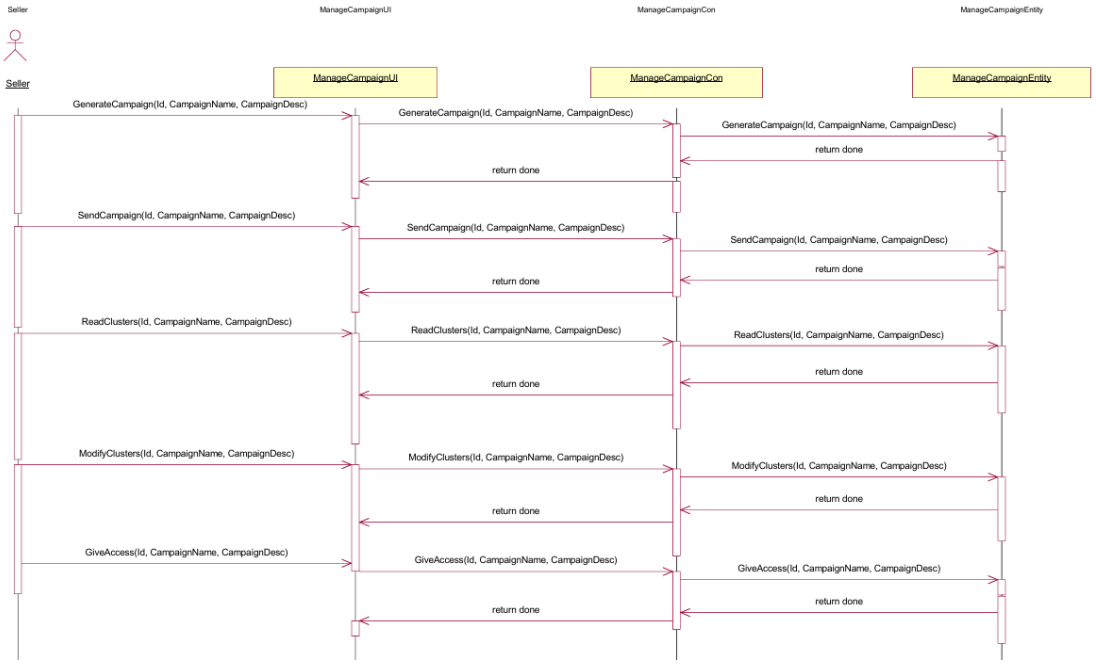


Figure C.5.7 Manage Campaign Sequence Diagram

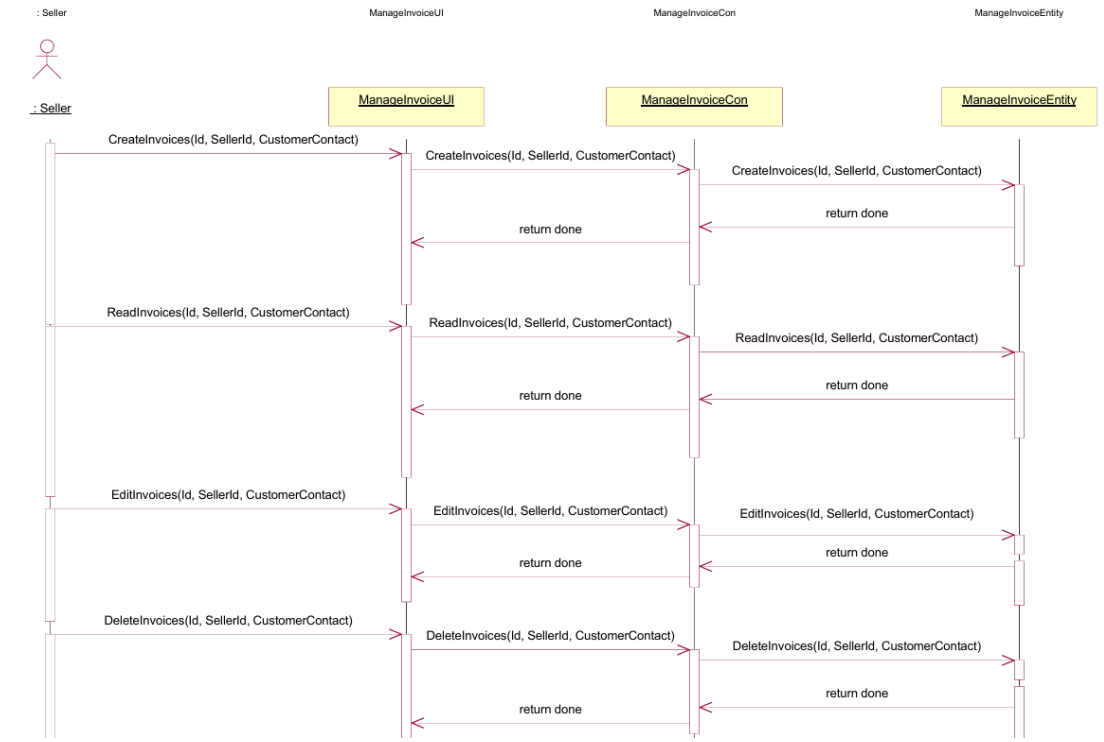


Figure C.5.8 Manage Division Sequence Diagram

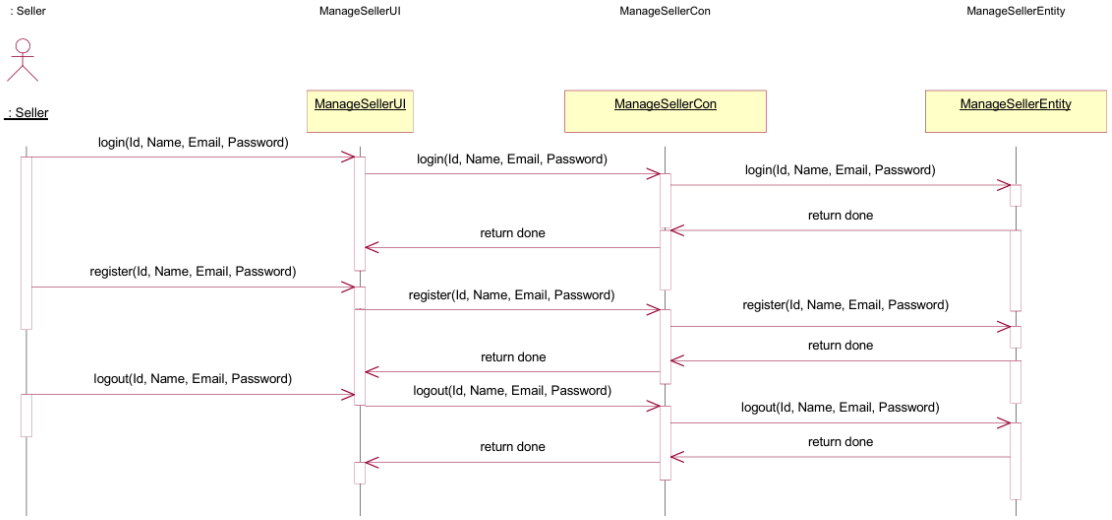


Figure C.5.9 Manage Seller Sequence Diagram

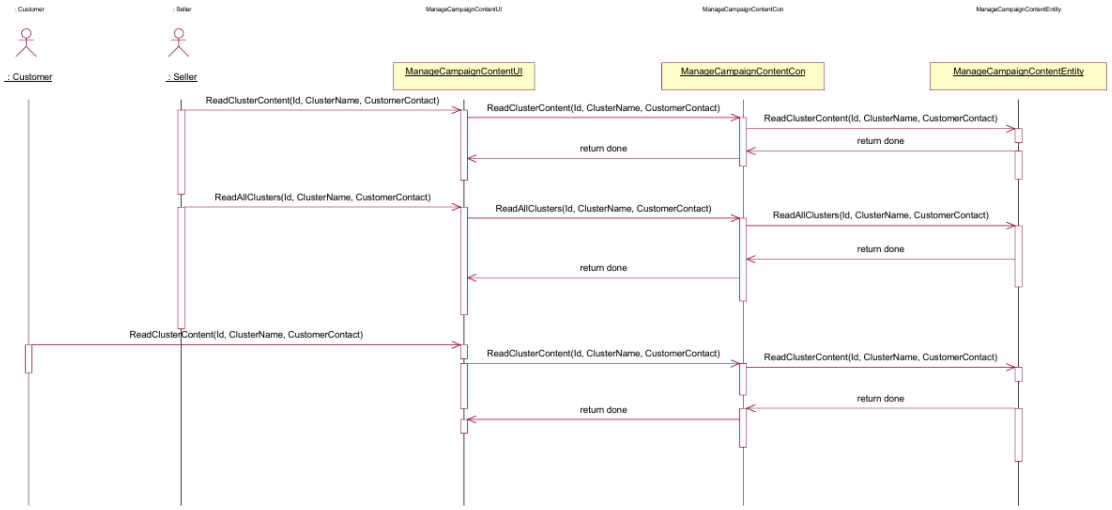


Figure C.5.10 Manage Campaign Content Sequence Diagram

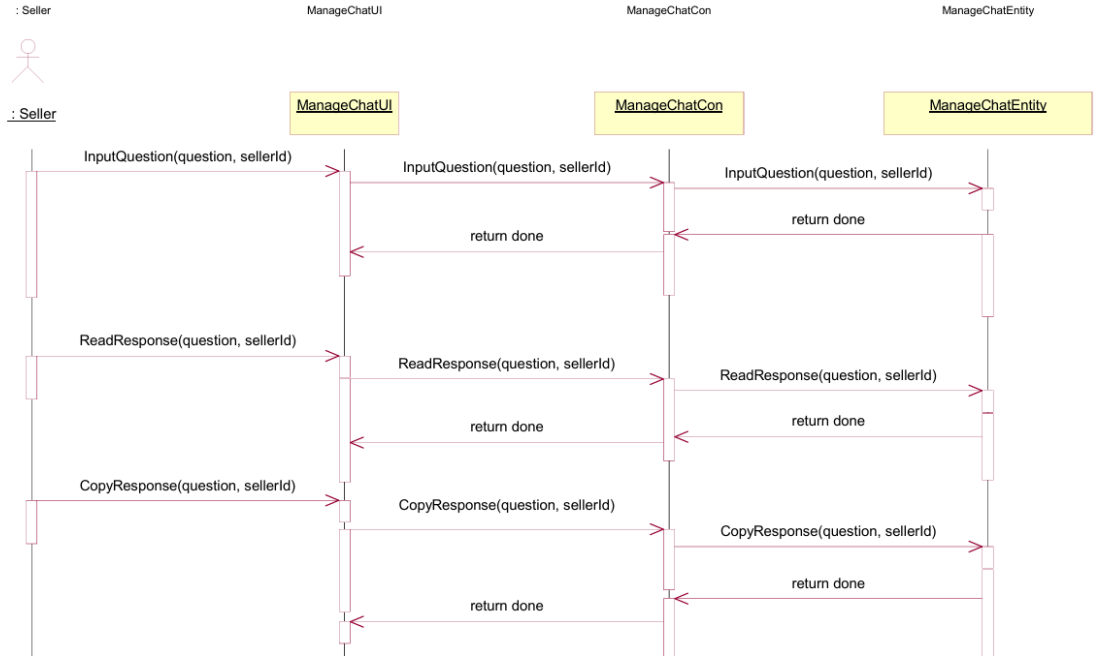


Figure C.5.11 Manage Chat Sequence Diagram

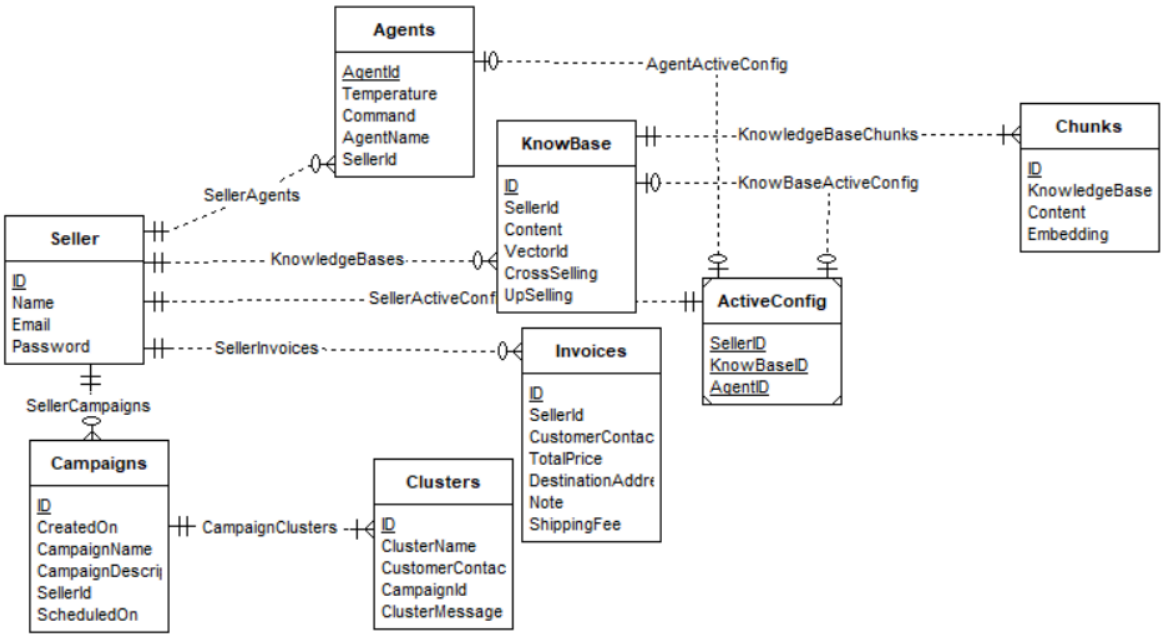


Figure C.5.12 Integrated Database System Entity Relational Diagram

D. VERIFICATION DEMONSTRATION AND PROOF OF DESIGN PROCESS

I. Home Page and Authentication

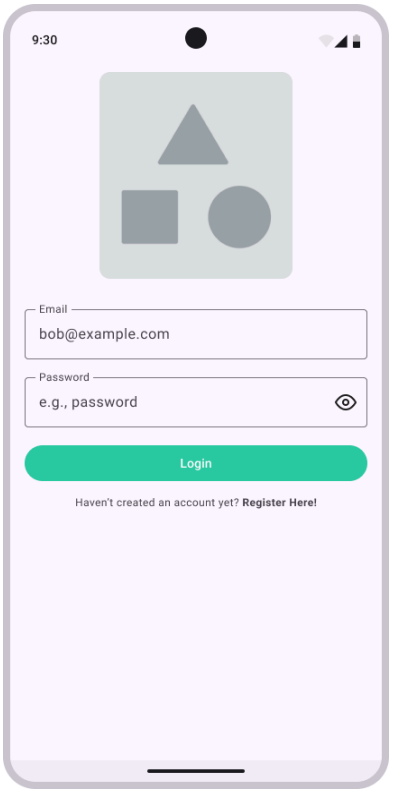


Figure D.I.1. Login page

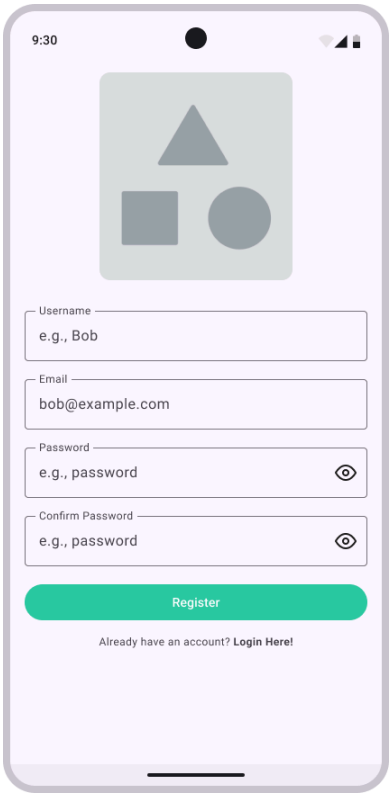


Figure D.I.2. Register page

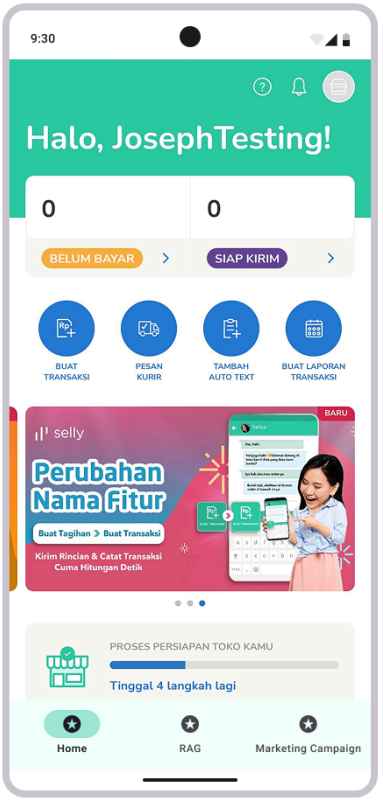


Figure D.I.3. Home Page

II. Auto Text RAG

Simulation Scenario:

The system is tested using a knowledge base containing 100 chunks from a simulated store information. Each chunk is set to explain different sectors or aspects of the business, creating entirely independent chunks of information from one another.

The whole knowledge base is separated into chunks by a predefined character that marks the heading of a new chunk. In this case, a markdown heading is detected as a heading for each chunk. For example, a possible input is:

“””

Company Overview

Company is from Jakarta and company sell phones. Company call themself ERAA. ERAA found on 1969 by Pter Grivin, Mac mcAI, Kent Uck Yzee, Friega Fast Chicken and nobody more.

Financial Performance

In 2025, company did lots of sales. Many domestic and even less foreign. Domestic get 10 million dollar net sales but foreign only get 5 million dollars net sales. Company still see growth from last year, 500% increase in sales revenue. Analysts optimistic on company.

Market Expansion

Over the age, company expand to countries, continents, nations, everywhere. Company store growth views 50000 stores across 9999 countries, 199999 continents, and 33333 nations as of Septembuary 1000. Expansion not only in store numbers but also quality, company put more flowers in front store people happy and go to store often and buy things more.

Products & Services

Since start, company sold phones, marketed as Rocks since the mid-980s. Since then, company expand product to various devices, starting the discontinued pods that can make sounds, and later the realer phones and bigger phone for fun. Company also

sold other products it categorizes "Can wear, Put in home and Can wear small", such as the no time Watch, no watch TV, pods sounds, pods at home, and ARVR.

Recent Developments
On Novemgust 1, 2100, Company announced buy all of Pixelguy, another company people like for tool to make image different on company native tool. Company had make people see Pixelguy good on showoff, also in it is make people know term Pixelguy Pro sitting tool of the century in 2000 for tool innovation in making self adapt to data and not real smart. In the flex, Pixelguy yapped that they too sigma to make changes even when bought by Company.
...
”””

From the input above the following output is as follows:
Chunk 1:
Company Overview
Company is from Jakarta and company sell phones. Company call themself ERAA. ERAA found on 1969 by Pter Grivin, Mac mcAI, Kent Uck Yzee, Friega Fast Chicken and nobody more.

Chunk 2:
Financial Performance
In 2025, company did lots of sales. Many domestic and even less foreign. Domestic get 10 million dollar net sales but foreign only get 5 million dollars net sales. Company still see growth from last year, 500% increase in sales revenue. Analysts optimistic on company.

Chunk 3:
Market Expansion
Over the age, company expand to countries, continents, nations, everywhere. Company store growth views 50000 stores across 9999 countries, 199999 continents, and 33333 nations as of Septembuary 1000. Expansion not only in store numbers but also quality, company put more flowers in front store people happy and go to store often and buy things more.

Chunk 4:

Products & Services

Since start, company sold phones, marketed as Rocks since the mid-980s. Since then, company expand product to various devices, starting the discontinued pods that can make sounds, and later the realer phones and bigger phone for fun. Company also sold other products it categorizes "Can wear, Put in home and Can wear small", such as the no time Watch, no watch TV, pods sounds, pods at home, and ARVR.

Chunk 5:

Recent Developments

On Novemgust 1, 2100, Company announced buy all of Pixelguy, another company people like for tool to make image different on company native tool. Company had make people see Pixelguy good on showoff, also in it is make people know term Pixelguy Pro sitting tool of the century in 2000 for tool innovation in making self adapt to data and not real smart. In the flex, Pixelguy yapped that they too sigma to make changes even when bought by Company.

...

Each chunk is stored as a chunk object on the PostgreSQL Relational Database. At the same time, the chunk content is also sent to the embedding model API to be represented into vectors. These vectors are then stored with PgVector linked to the appropriate chunk where the vector originates from.

Aside from knowledge base, the agent information is also required. The system is tested using a command to tell customers of the business and try to add questions at the end of each response of what the customer might want to ask further based on the information they asked. An example of the command is as follows:

“””

You are an intelligent and interactive marketing agent for ERAA. When a user asks a question, provide a detailed answer. Then, suggest a follow up question to entice the user in exploring the company further. Tailor these follow-ups based on the user’s

original question, making them progressively more insightful or expanding into related areas. Keep the interaction engaging and informative
“””

Seller’s query is then simulated by creating a question that can be answered using the information inputted previously on the knowledge base. An example of said question is as follows.

“””
"What steps has ERAA taken to expand its market presence?"

“””

The customer’s question is then sent to the embedding model API. The vector representation is produced from the API request and a pgVector operation on vector similarity is done using the negative inner product operator.

The similarity measurement will return the top 3 most similar chunks and include them on the prompt as well as the user question and command to the Large Language Model API.

Simulation Results:

Chunk Similarity Results

Chunk Heading	Similarity Rate
Market Expansion	0.92
Recent Developments	0.85
Company Overview	0.67
Financial Performance	0.43
Products and Services	0.35

LLM Response Result

“”””

ERAA has taken several steps to expand its market presence. Recently, the company has developed the number of retail locations and its geographical coverage, with 50000 stores across 9999 countries worldwide as of Septembuary 1000.

ERAA expansion not only done in quantity, but also quality. ERAA will continue to improve existing stores as they undergo more flowering. Flowering makes people happy so stores receive more customers and more sales.

Would you like to learn more about how the expansion has impacted ERAA’s finances?

“”””

Discussion and explanation based on the simulation results

The simulation shows that the system functions effectively across its main components:

- Embedding Model API Call successfully returned the vector representation of each chunk and query,
- Negative Inner Product similarity measurement has succeeded in ranking the chunks based on the most similar to least similar to the question’s vector embeddings,
- The Large Language Model API Call has successfully resulted in the response answering the user’s question properly while also abiding to the command and question context prepared
- Sellers were able to copy the message and directly send it to the user as it has already been crafted so that the customer can understand the result as if they are talking directly to the seller.

Proof of Design Process

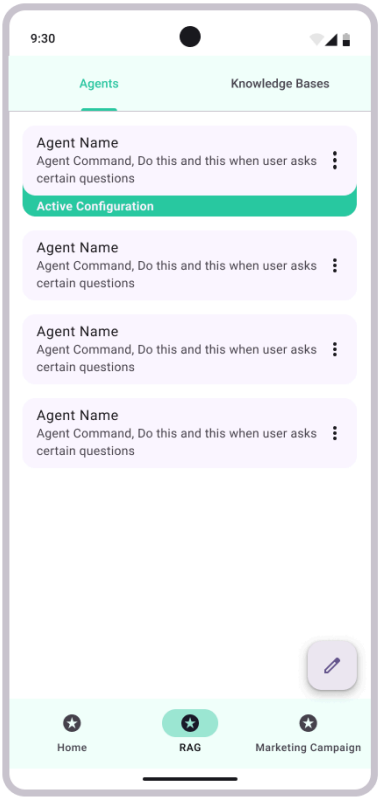


Figure D.II.1. RAG Agents Page

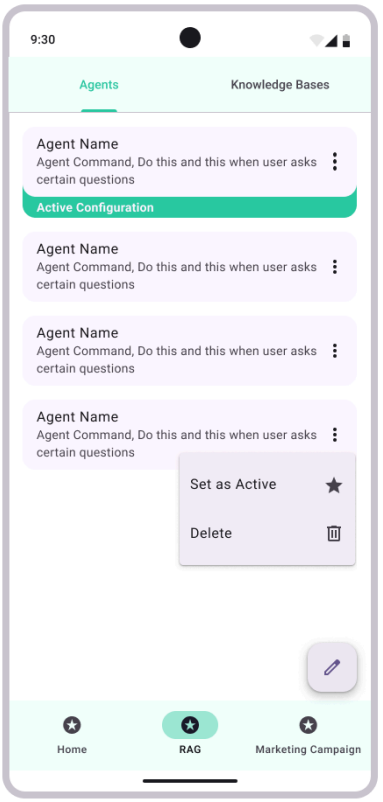


Figure D.II.2. RAG Agents Expanded Page

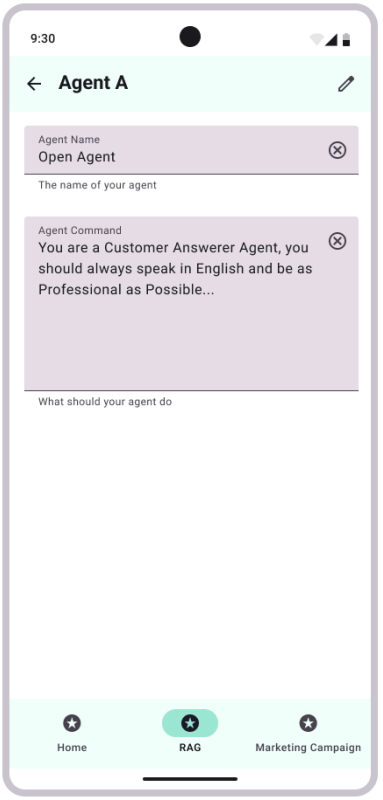


Figure D.II.3. RAG Agent Item Page

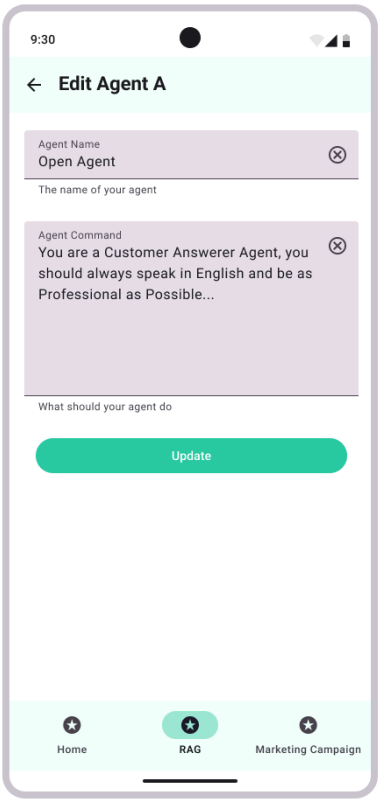


Figure D.II.4. RAG Update Agent Form Page

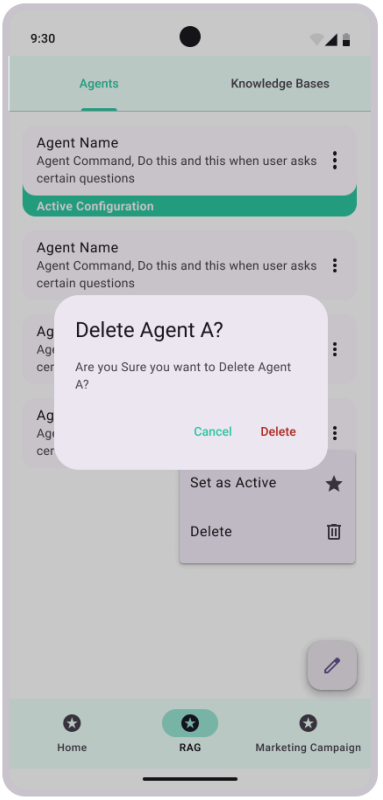


Figure D.II.5. RAG Delete Agent Dialog Page

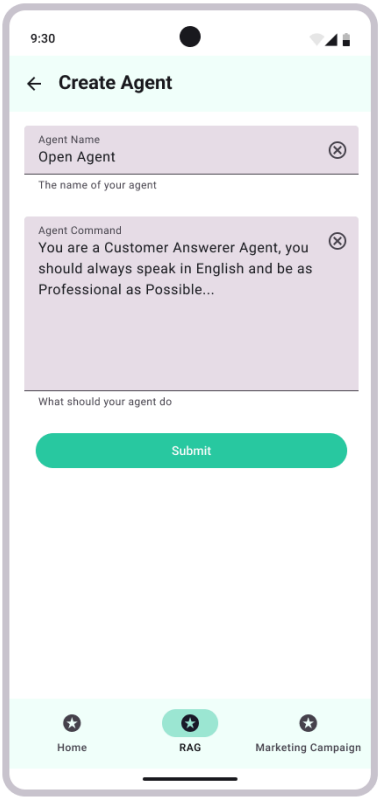


Figure D.II.6. RAG Create Agent Dialog Page

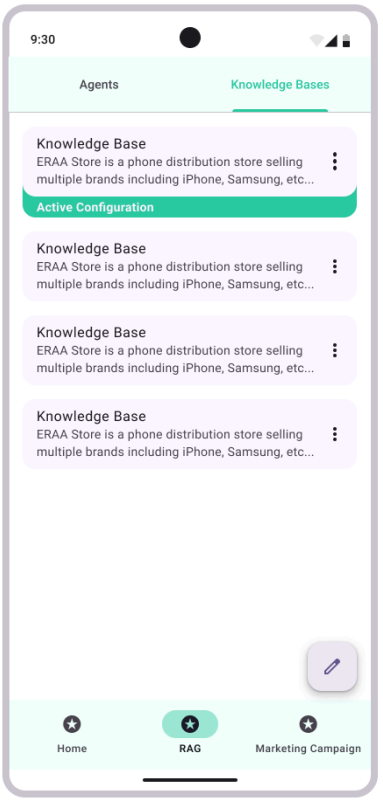


Figure D.II.7. RAG Knowledge Base Page

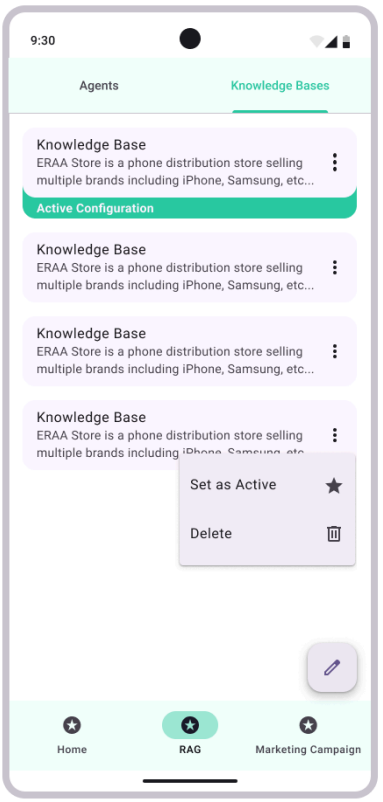


Figure D.II.8. RAG Knowledge Base Expanded Page

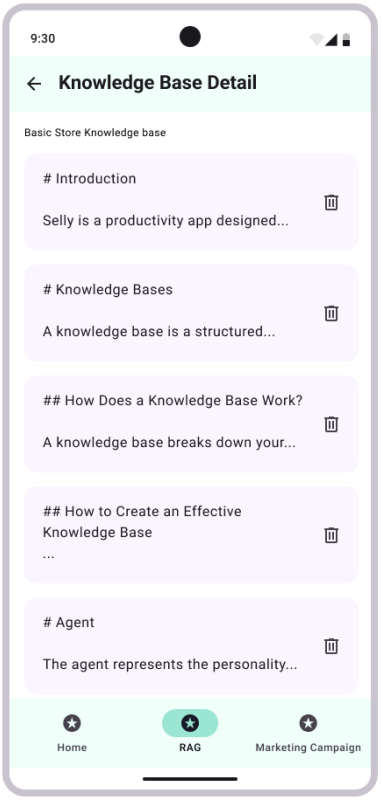


Figure D.II.9. RAG Read Knowledge Base Detail Page

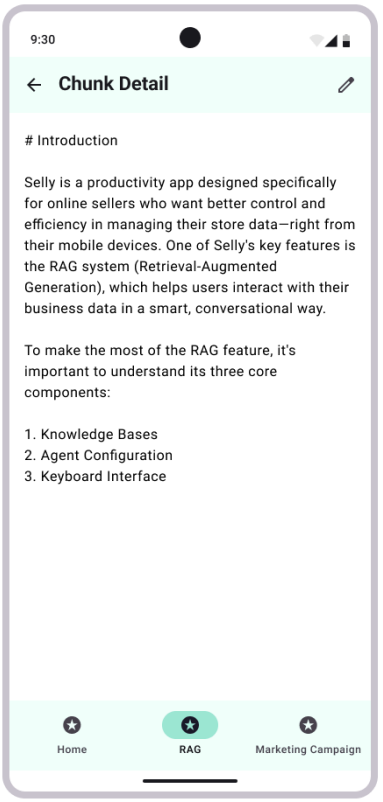


Figure D.II.10. RAG Read Chunk Detail Page

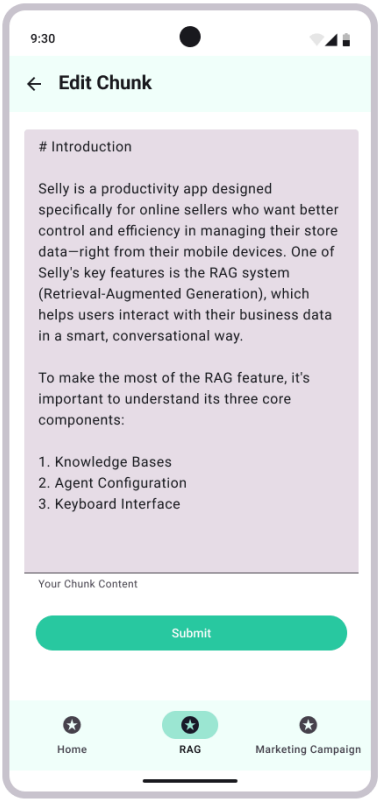


Figure D.II.11. RAG Edit Chunk Page

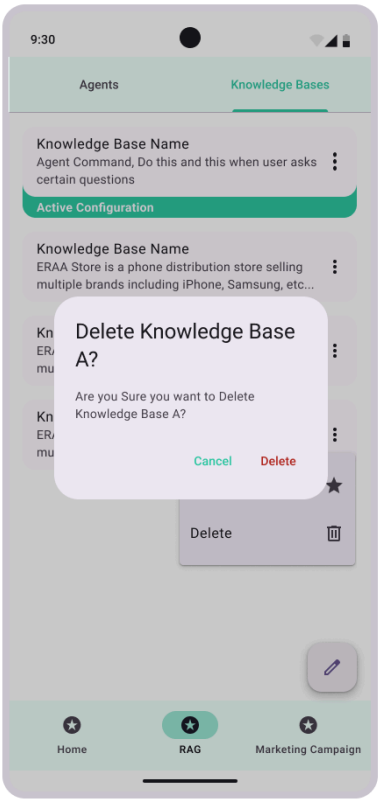


Figure D.II.12. RAG Delete Knowledge Base Item Page

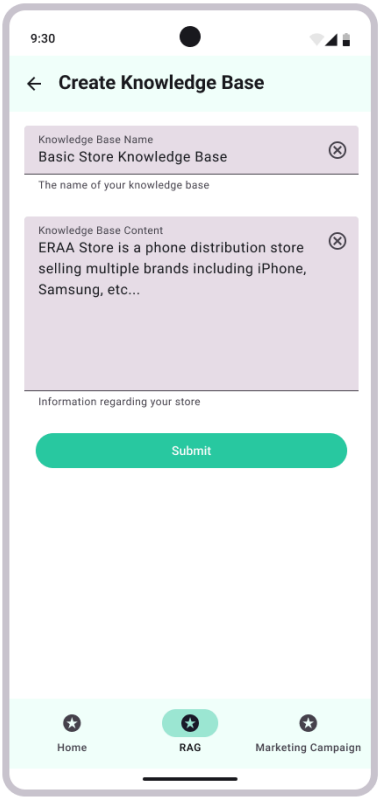


Figure D.II.13. RAG Create Knowledge Base Item Page

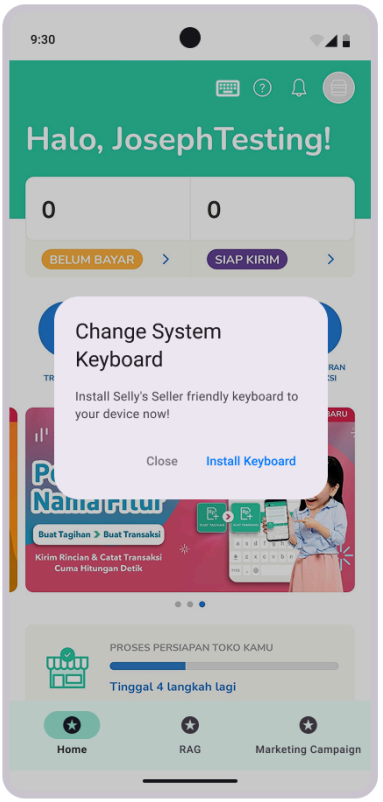


Figure D.II.14. Change System Keyboard Dialog

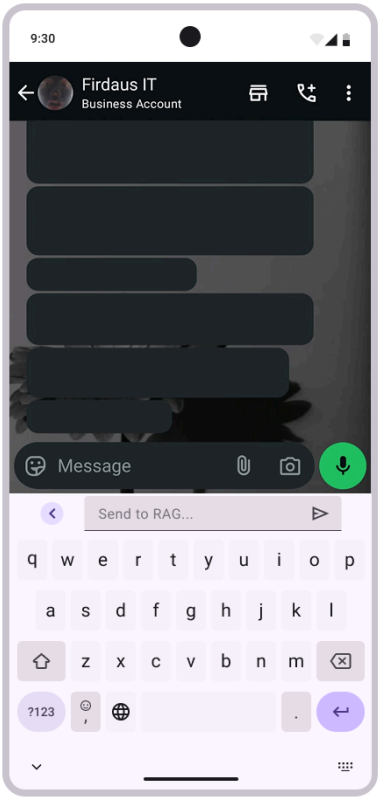


Figure D.II.15. RAG Keyboard Initial Page

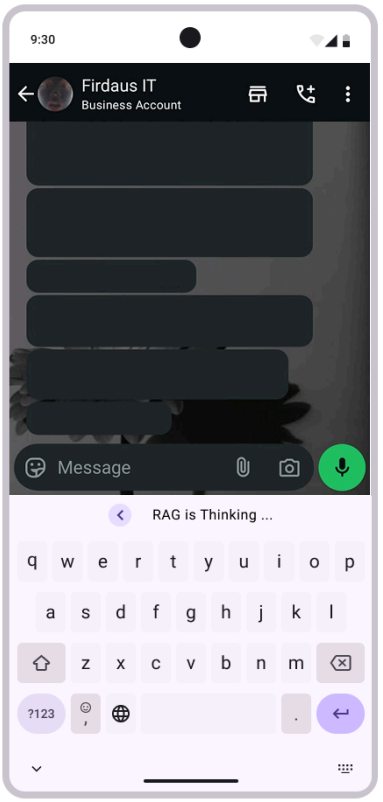


Figure D.II.16. RAG Keyboard Loading Page

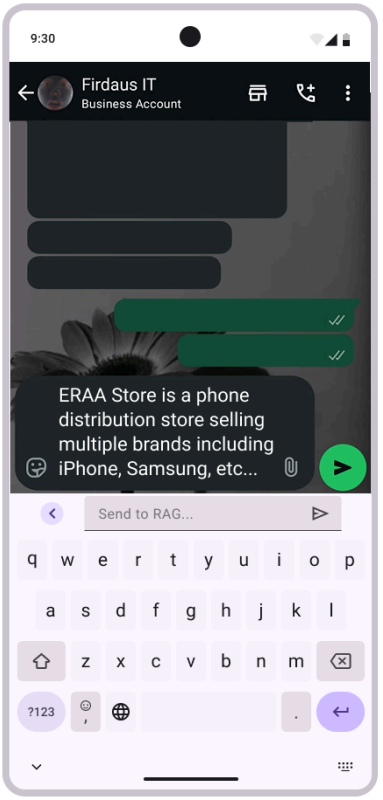


Figure D.II.17. RAG Keyboard Success Page

III. Personalized Marketing Campaign Generator

Simulation Scenario:

The system is tested using a dataset of 100 sample invoices from simulated customers. Each customer has a different purchase frequency, monetary value, and recency profile. K-Means algorithm is used to cluster them into three segments: Loyal, Moderate, and At-Risk.

For each segment, the system calculates behavioral summaries (e.g., average orders per month, days since last purchase), then sends that summary to the LLM API (e.g., OpenAI) to generate a suggested promotional offer. The seller reviews each suggestion before final approval.

Simulation Results:

Segment	Consumer Behavior Summary	LLM Suggested Offer	Seller Action
Loyal	3.1 orders/month, high consistency	"10% discount to maintain loyalty"	Accepted as-is
Moderate	1.4 orders/month, medium spend	"Free shipping to encourage activity"	Slightly modified text
At-Risk	No purchases in past 45 days	"30% discount with urgency message"	Increased urgency tone

The simulation shows that the system functions effectively across its main components:

- K-Means clustering successfully grouped customers based on real transaction patterns.
- Behavioral summaries were correctly calculated/generated from the invoice data.
- The LLM returned logical, context-aware suggestions based on each summary.
- Sellers were able to review and finalize offers with minimal adjustment needed.

This confirms that the end-to-end flow from data to AI suggestion to seller decision is working as intended. It also demonstrates that the AI is acting as a reliable co-pilot in the decision-making process, not replacing human judgment but supporting it.

Proof of Design Process

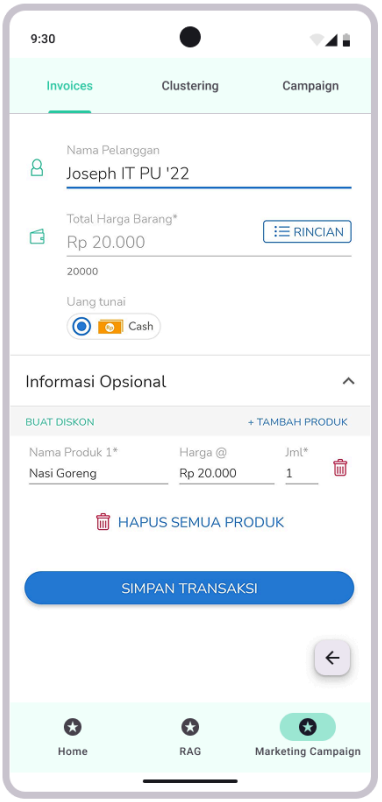


Figure D.II.1. Add New Invoice Page

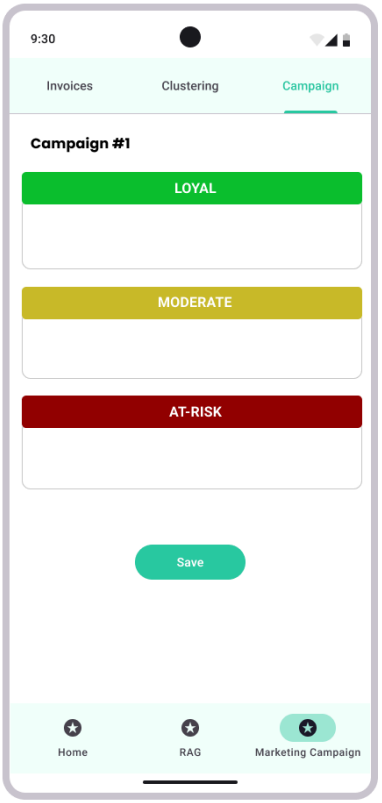


Figure D.II.2. Campaign Add Page

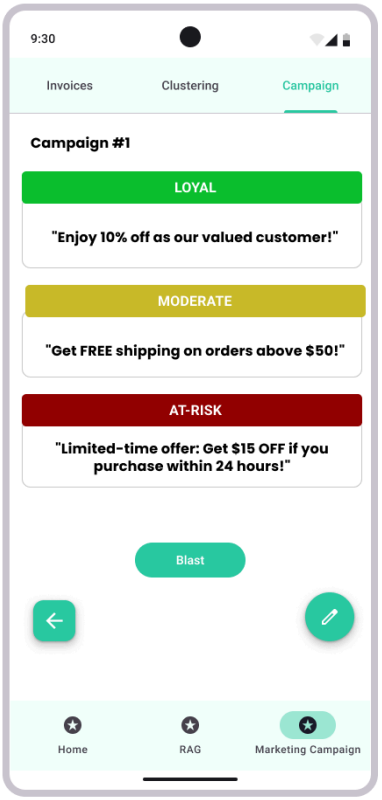


Figure D.II.3. Campaign Detail Page

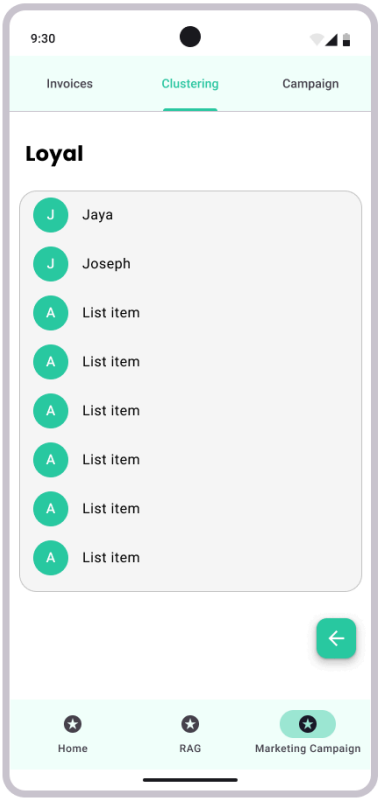


Figure D.II.4. Cluster Final Group Output Page

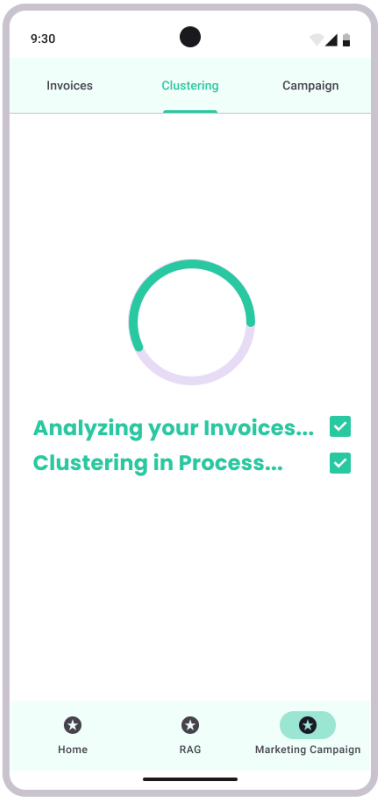


Figure D.II.5. Clustering Process Page

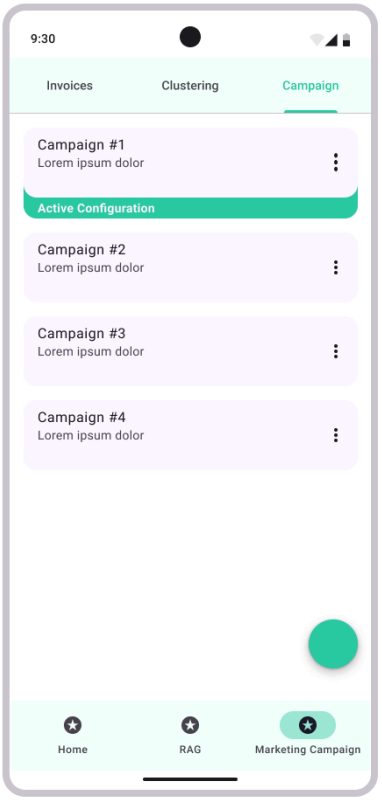


Figure D.II.6. Initial Campaign Page

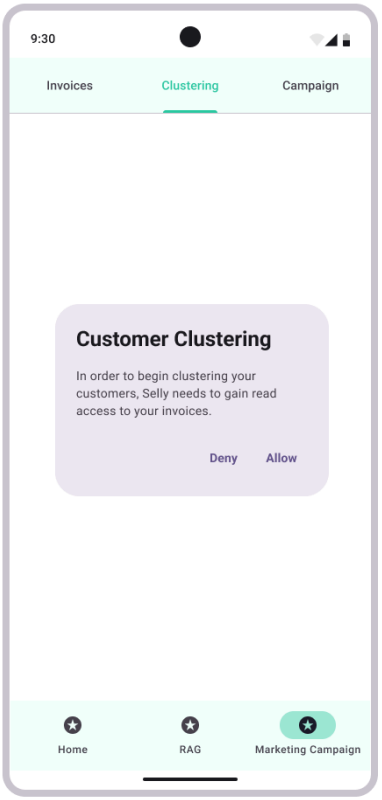


Figure D.II.7. Initial Clustering Page

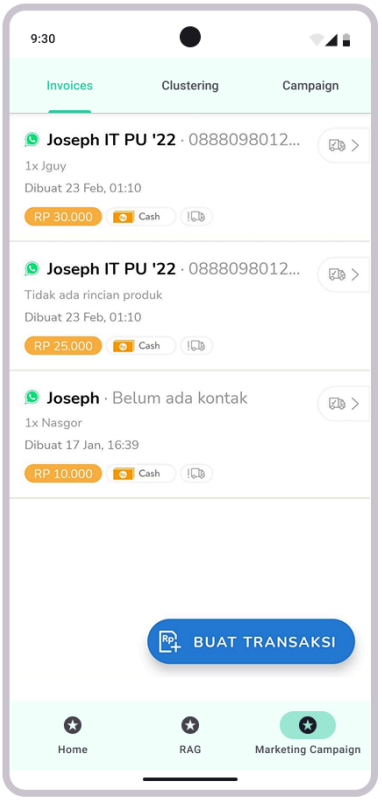


Figure D.II.8. Invoice List Page

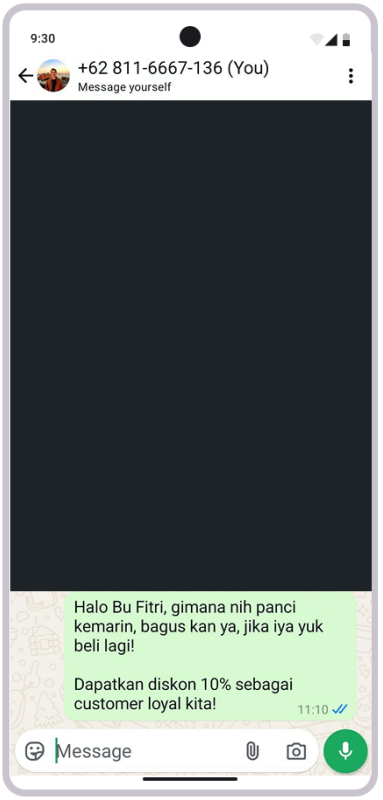


Figure D.II.9. Campaign Implementation Page

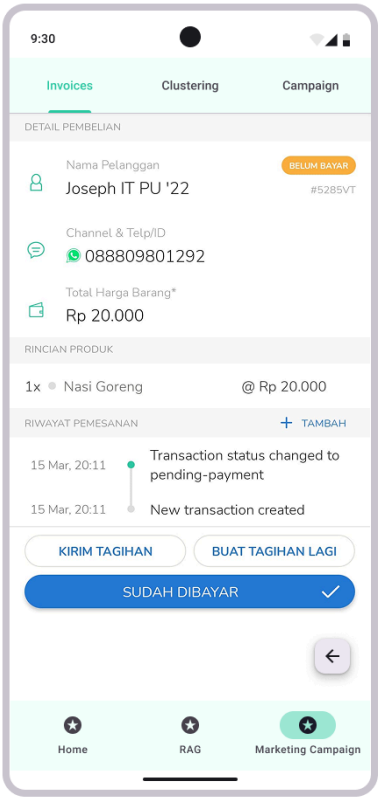


Figure D.II.10. Read (Access) Invoice Page

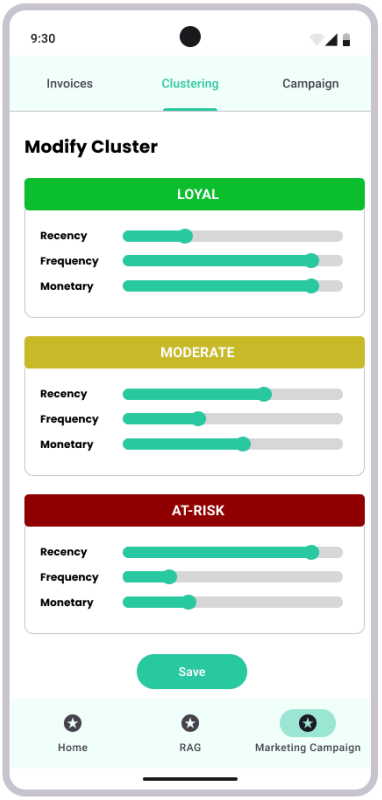


Figure D.II.11. RFM K-Means Clustering Modify Page

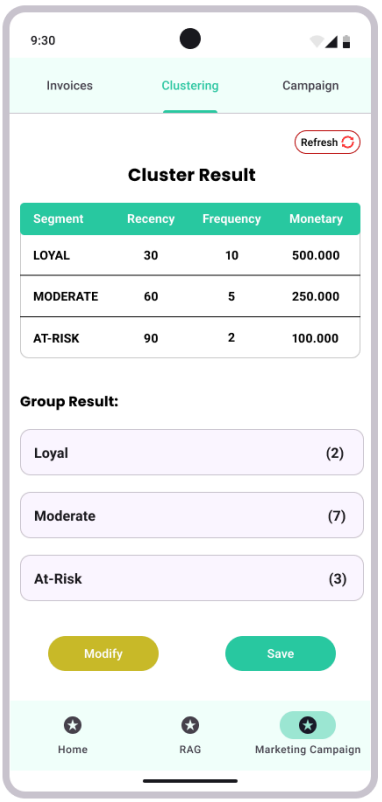


Figure D.II.12. RFM K-Means Clustering Result Page

E. STANDARDS USED

1. Data Formats

- JSON: Used for communication between backend services and the LLM (e.g., input/output payloads).
- CSV: Used for importing/exporting invoice or transaction history data.
- Datetime Format: ISO 8601 (YYYY-MM-DD) used for consistency in handling purchase dates and recency analysis.

2. Algorithmic Standards

- RFM Scoring Convention: Standard RFM method for Recency, Frequency, Monetary value analysis.
- K-Means Clustering: Unsupervised machine learning algorithm for segmentation.
- Vector Similarity Measurement: Negative Inner Product operation of Two Vectors.

3. Prompt Engineering Practices

- Marketing prompt format follows the standard structure: clear role instruction, concise input context, and expected output style.
- RAG Prompt is structured starting from the command, topmost similar chunks of information, and customer query.
- Avoids ambiguous phrasing and hallucination triggers per LLM prompt best practices.

4. Message Channels

- UTF-8 Encoding: Ensures compatibility for multilingual message support (e.g., Bahasa Indonesia).
- WhatsApp Template Compliance: Message formats are kept structured.

5. APIs

- RESTful API conventions: Standard HTTP methods (GET, POST) and status codes used for internal API communication.
- LLM e.g. openAI API: Complies with OpenAI's official usage documentation and token limits.
- Embedding e.g. VoyageAI API: Complies with VoyageAI's official usage documentation and token limits.

6. Labeling Conventions

- Customer segments are labeled using standard business-friendly terms: “Loyal,” “Moderate,” and “At-Risk.”
- Transaction fields follow common naming standards (e.g., order_id, customer_id, total_amount).

a. IMPLEMENTATION AND TESTING PLANS

I. Gantt Chart

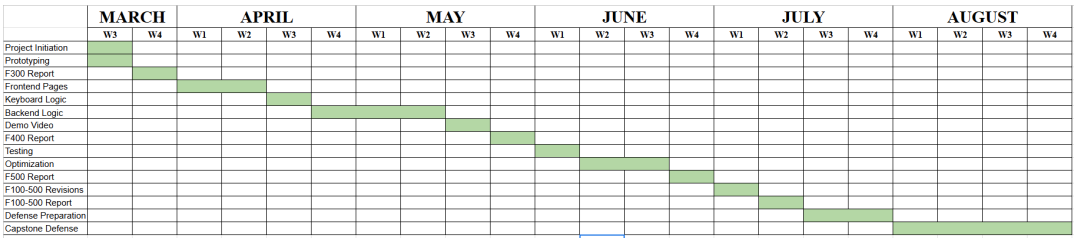
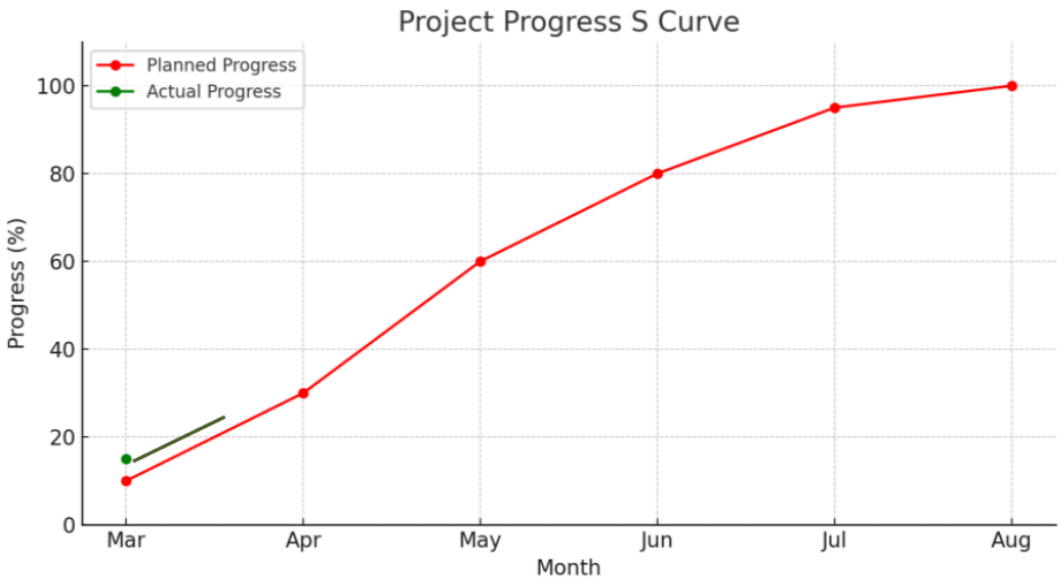


Figure F.I. Project Weekly Gantt Chart Incorporating Schedule and Work Dependencies

II. S-Chart



Description	Mar	Apr	May	Jun	Jul	Aug
Cum. Planned Progress	10.0%	30.0%	60.0%	80.0%	95.0%	100.0%
Cum. Actual Progress	15.0%					
Variance	5.0%					

Figure F.II. S-Curve Chart for Project Progress

No.	Specification	Test Method	Amount to be Measured
1.	Prototyping	Screenshots and mockup review	Visual fidelity (≥80% match with design)
2.	Frontend Pages	Manual UI walkthrough	All key pages load without crash, navigable
3.	Keyboard Logic	Input simulation	Key event responses
4.	Backend Logic	Test the response	Success response rate
5.	Demo Video	Play through entire video to ensure flow, narration, and feature coverage	Full run duration from intro to outro
6.	Testing	Unit tests and manual QA sessions	≥90% test case pass rate
7.	Optimization	Feature improvement before/after optimization	Smoothness rate
8.	F100–F500 Reports	Supervisor feedback on drafts, AI check	≥90% quality score, <5% AI detected
9.	Defense Preparation	Presentation dry-run, timing, and QnA practice	Delivery time & QnA coverage

Table F.II Details Breakdown of the S-Curve Progress

F. REFERENCES

Enevoldsen, K., Chung, I., Kerboua, I., Kardos, M., Mathur, A., Stap, D., Gala, J., Siblini, W., Krzemiński, D., Winata, G. I., Sturua, S., Utpala, S., Ciancone, M., Schaeffer, M., Sequeira, G., Misra, D., Dhakal, S., Rystrøm, J., Solomatin, R., … Muennighoff, N. (2025). *MMTEB: Massive Multilingual Text Embedding Benchmark* (No. arXiv:2502.13595). arXiv. https://doi.org/10.48550/arXiv.2502.13595

Ezra, P. N., Egedegbe, F. E., Nwafor, C. N., Obasi, I. N., & Okonta, C. A. (2024). Project Management Using Critical Path Method (CPM) and Project Evaluation and Review Technique (PERT). *International Journal Of Mathematics And Computer Research*, *12*(12), Article 12. https://doi.org/10.47191/ijmcr/v12i12.03

Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., & Bowman, S. R. (2023). *GPQA: A Graduate-Level Google-Proof Q&A Benchmark* (No. arXiv:2311.12022). arXiv. https://doi.org/10.48550/arXiv.2311.12022

Vinerean, S., & Opreana, A. (2024). *Artificial Intelligence and its Role in Personalized Marketing for Effective Customer Engagement*.